

Operating System Concepts

A. Silberschatz, P. B. Galvin, and G. Gagne, 10th edition

Chapter 10: Virtual Memory

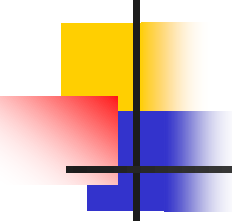
Mei-Ling Chiang, Professor

Dept. of Information Management

National Chi-Nan University, Taiwan, ROC



2025/12/04



Chapter 10: Virtual Memory

10.1 Background



10.2 Demand Paging



10.3 Copy-on-Write



10.4 Page Replacement



10.5 Allocation of Frames



10.6 Thrashing



10.7 Memory Compression



10.8 Allocating Kernel Memory

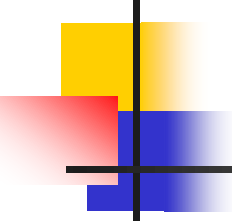


10.9 Other Considerations



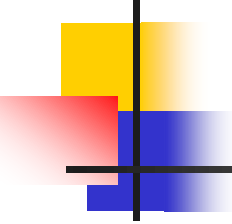
10.10 Operating-System Examples





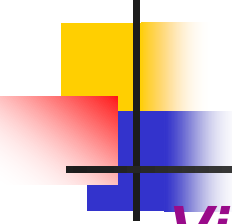
10.1 Background

- **There are many cases where not entire program is needed:**
 - **Code to handle unusual error conditions is almost never executed**
 - **Arrays, lists, tables are often allocated more memory than they actually need**
 - **Certain options and features of a program may be used rarely**
- **Even the entire program is needed, it may not all be needed at the same time**



Background (Cont.)

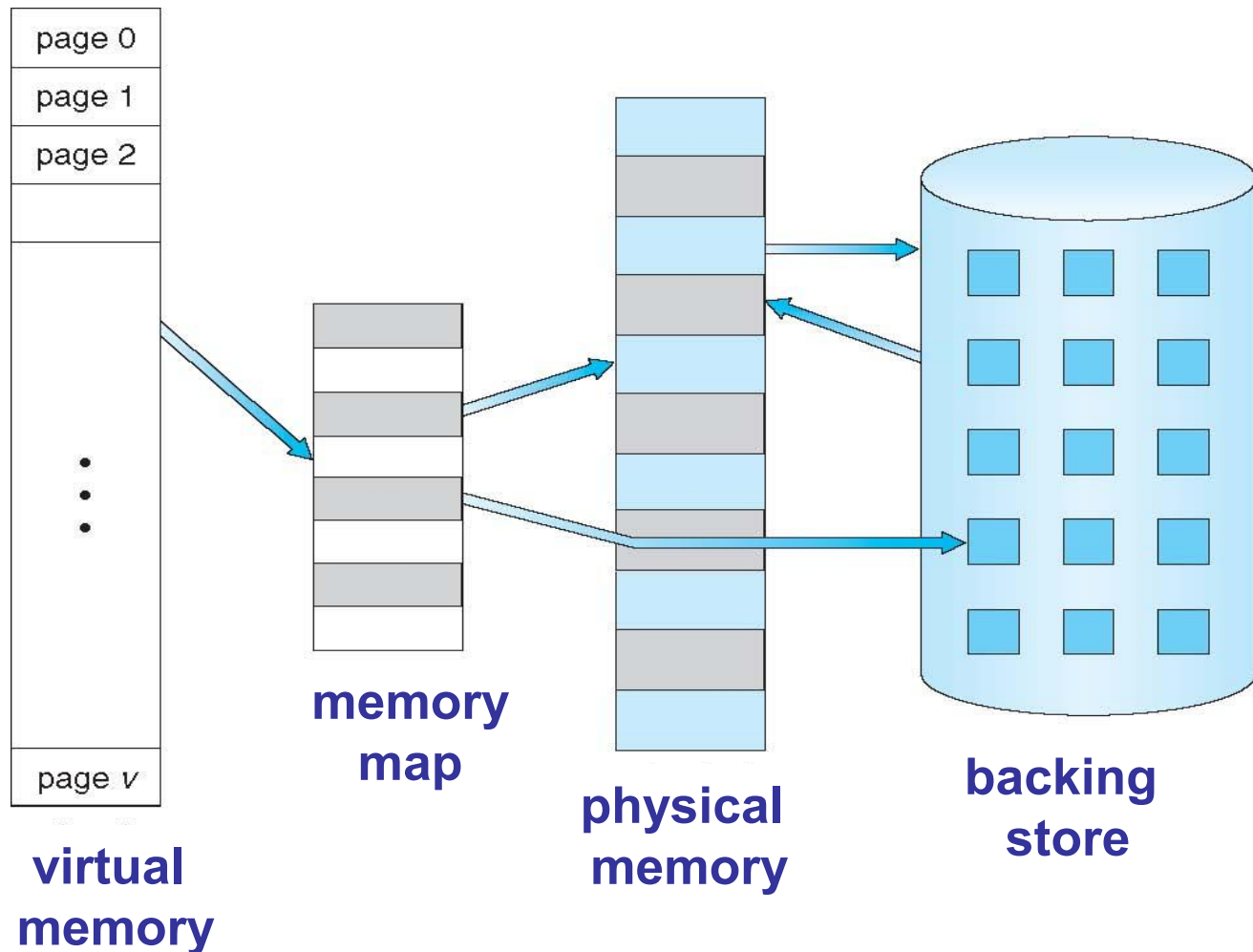
- **Benefits of the ability to execute a program that is only partially in memory**
 - Have a very large *virtual* address space
 - Not constrained by the amount of available physical memory
 - **Higher multiprogramming degree**
 - increase CPU utilization and throughput
 - **Program runs faster**
 - less I/O would be needed to load or swap

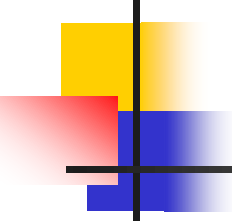


Virtual Memory

- ***Virtual memory*** is a technique that allows the execution of processes that are not completely in memory.
- ***Virtual memory*** – separation of user logical memory from physical memory.
 - Only part of the program needs to be in memory for execution.
 - Logical address space can therefore be much larger than physical address space.
 - Allows processes to share files easily and to implement shared memory.
 - Allows for more efficient process creation (copy-on-write).
- Virtual memory can be implemented via:
 - **Demand paging**

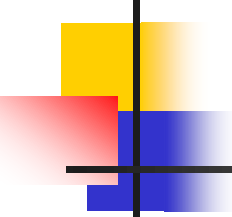
Virtual Memory That is Larger Than Physical Memory (Fig. 10.1)





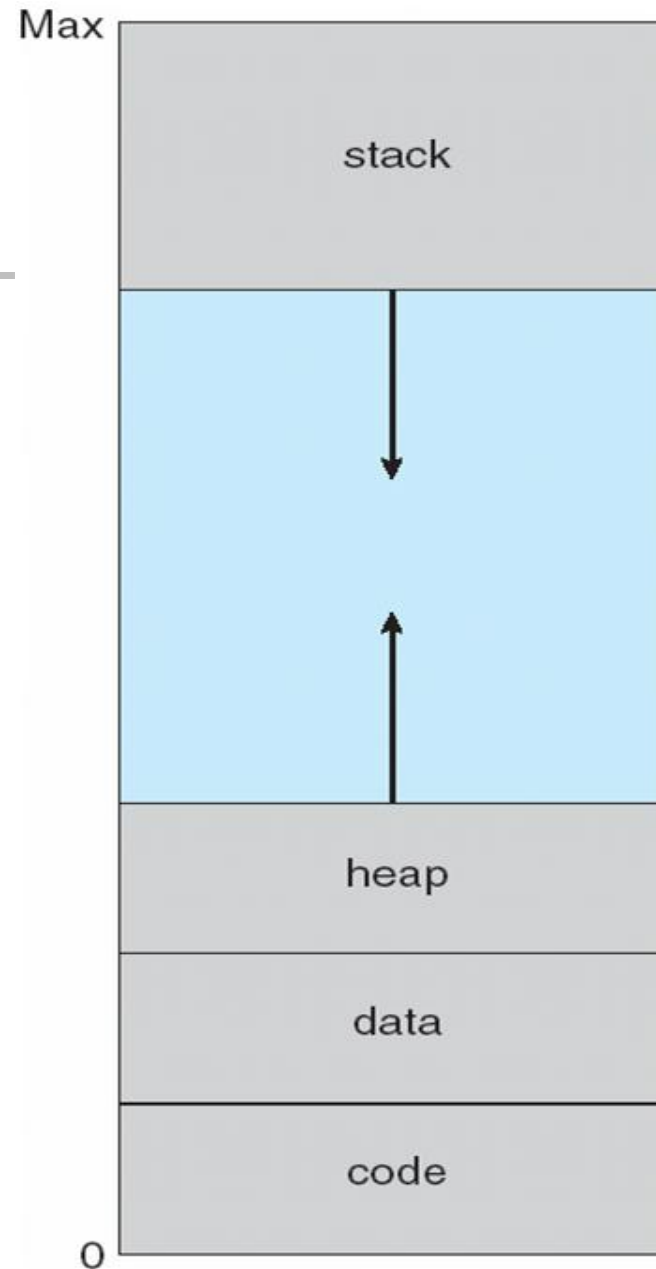
Background (Cont.)

- ***Virtual address space*** of a process refers to the logical (or virtual) view of how a process is stored in memory.
 - Typically, this view is that a process begins at a certain logical address – say, address 0 – and exists in contiguous memory.
 - Physical page assigned to a process may not be contiguous.
 - Up to the MMU to map logical pages to physical page frames in memory.

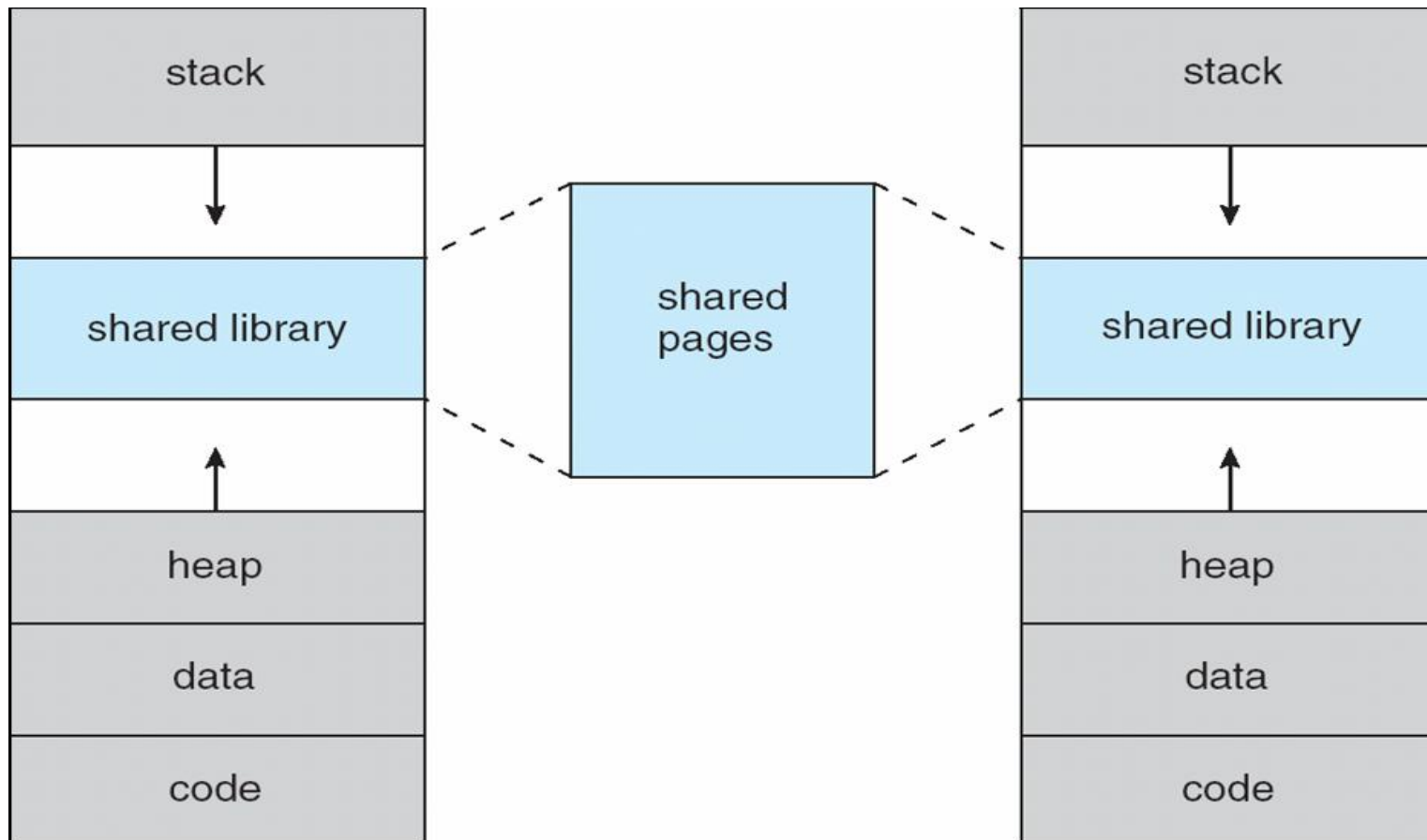


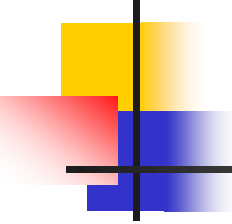
Virtual address space of a process in memory

(Fig. 10.2)



Shared Library Using Virtual Memory (Fig. 10.3)



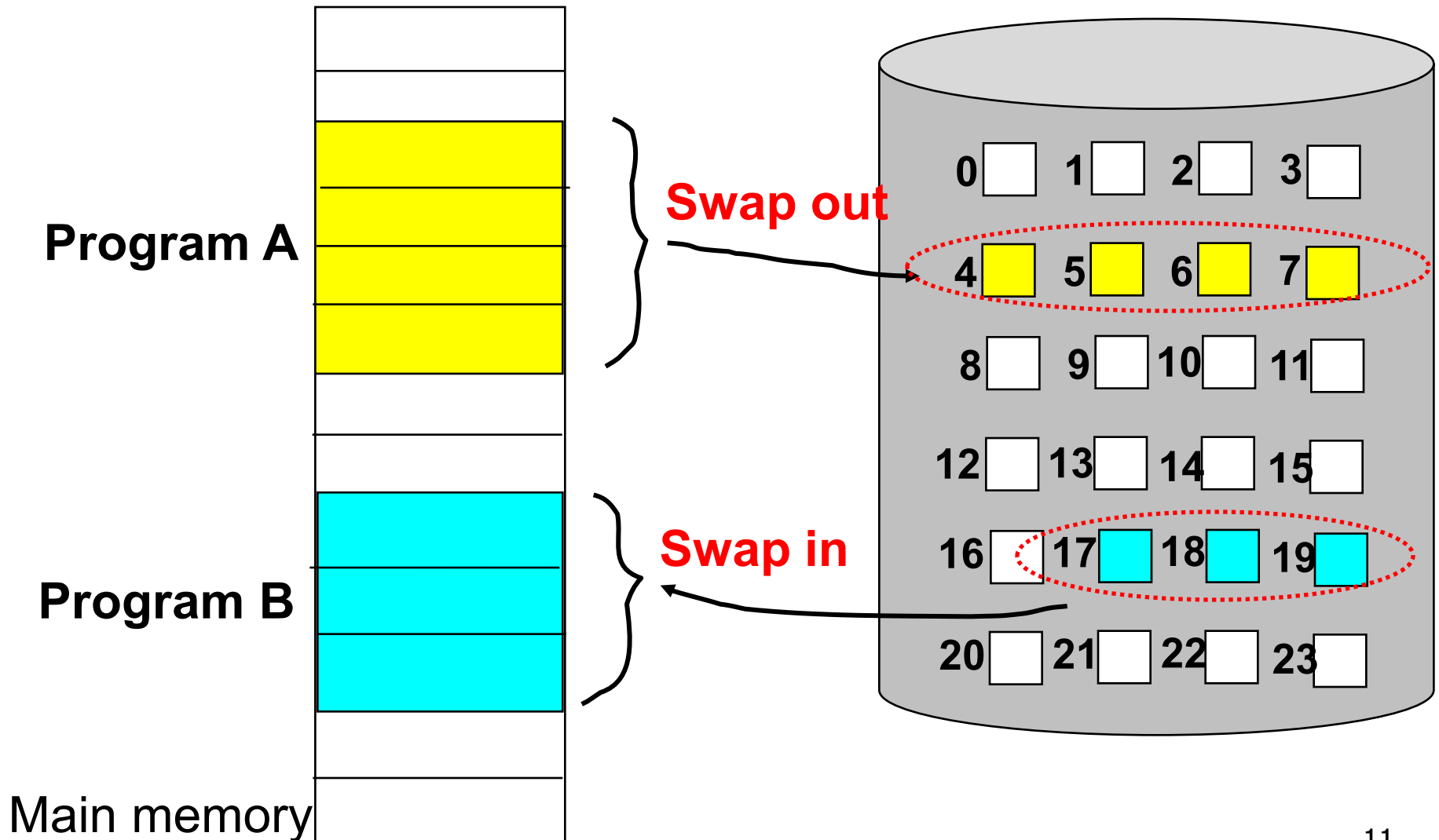


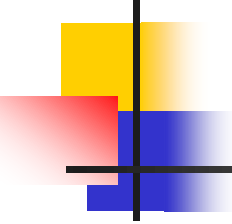
10.2 Demand Paging

- With demand-paged virtual memory, pages are only loaded when they are demanded during program execution; pages that are never accessed are thus never loaded into physical memory.
- Similar to a paging system with swapping
- Rather than swapping the entire process in to memory, we use a **lazy swapper**
 - A lazy swapper never swaps a page into memory unless that page will be needed
- **Pager** is the correct term (rather than **swapper**)

A Paging System With Swapping

Transfer of a paged memory to contiguous disk space

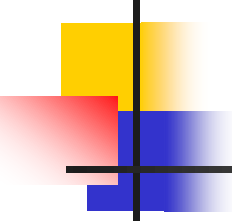




Demand Paging

- **10.2.1 Basic Concepts** 
- **10.2.2 Free-Frame List** 
- **10.2.3 Performance of Demand Paging** 





10.2.1 Demand Paging –Basic Concepts

- **Bring a page into memory only when it is needed.**
 - **Less I/O needed**
 - **Less memory needed**
 - **Faster response**
 - **More users**
- **Page is needed $\Rightarrow$ reference to it**
 - **invalid reference $\Rightarrow$ abort**
 - **not-in-memory $\Rightarrow$ bring to memory**

Demand Paging (Cont.)

- Requires hardware support:

- With each page table entry a **valid-invalid bit** is associated

- 1 $\Rightarrow$ legal, in-memory
- 0 (*invalid*) $\Rightarrow$ not-in-memory

- During address translation, if valid–invalid bit in page table entry is 0 $\Rightarrow$ **page fault**.

Frame #	valid-invalid bit
	1
	1
	1
	1
	0
⋮	
	0
	0

page table

Page table when some pages are not in main memory (Valid-Invalid Bit) (Fig. 10.4)

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

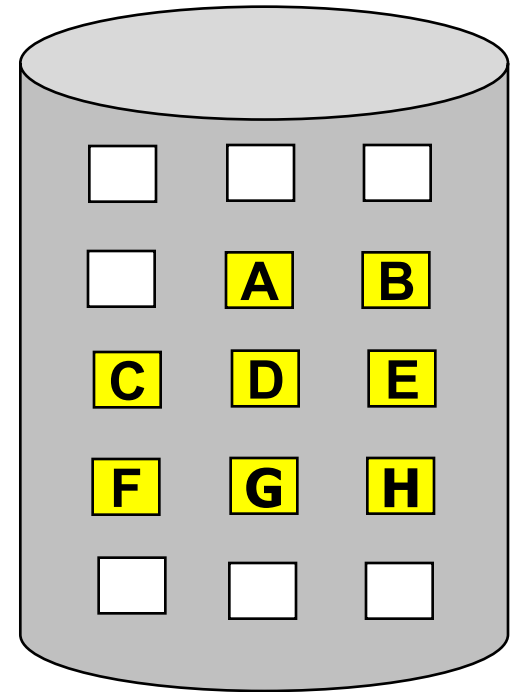
logical memory

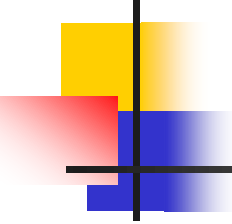
valid-invalid	
frame	bit
0	4 v
1	i
2	6 v
3	i
4	i
5	9 v
6	i
7	i

page table

0	
1	
2	
3	
4	A
5	
6	C
7	
8	
9	F
10	
11	
12	
13	
14	
15	

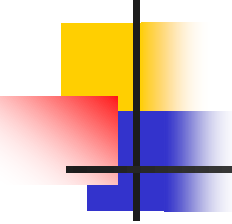
physical memory





Page Fault

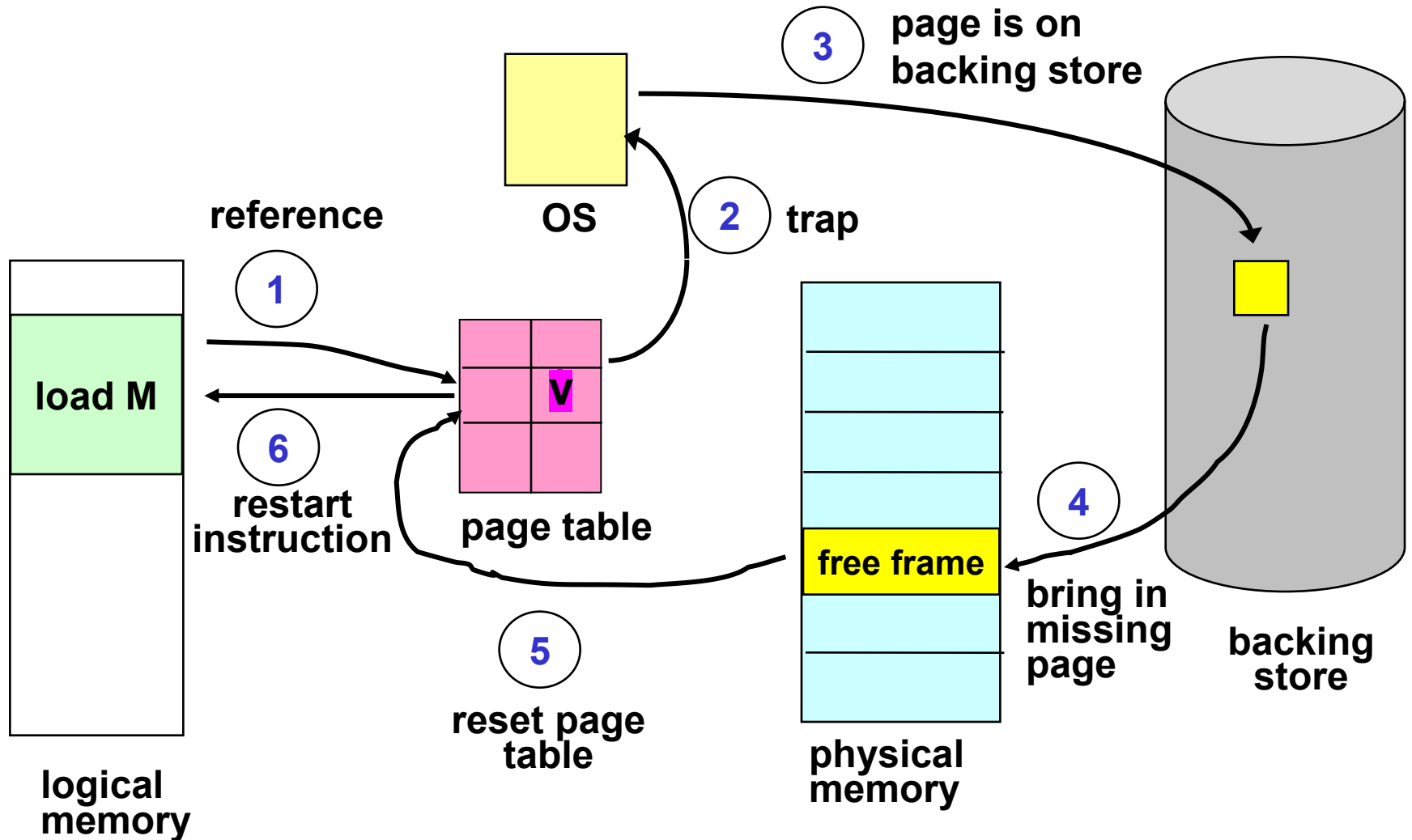
- Access to a page marked invalid causes a *page-fault trap*
 - OS (*page fault handler*) then reads the desired page into memory and restarts the instruction that was interrupted by the trap as though the page had always been in memory
- *Pure demand paging*
 - Never bring a page into memory until it is required.

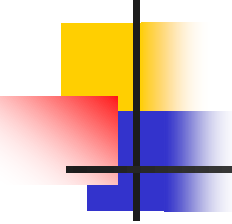


Handling a Page Fault

- If there is ever a reference to a page, first reference will trap to OS $\Rightarrow$ page fault
- OS looks at another table to decide:
 - Invalid reference $\Rightarrow$ abort.
 - Just not in memory.
- Get empty frame.
- Read the desired page into frame.
- Reset tables, valid bit = 1.
- Restart the faulted instruction

Handling a Page Fault (Fig. 10.5)





Demand Paging (Cont.)

- **Hardware to support demand paging:**
 - **Page Table:** mark an entry invalid
 - **Secondary memory** (a high-speed disk or NVM device) to hold those pages that are not present in main memory, (**swap device**, **swap space**)
- **Additional architectural constraints must be imposed.**
 - A crucial one is the need to be able to restart any instruction after a page fault.

Restarting Instruction After Page Fault

- **Example :**

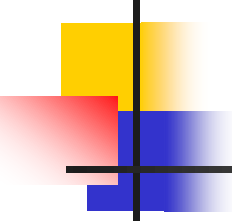
$C \leftarrow A + B$

- 1. Fetch and decode the instruction (ADD)**
- 2. Fetch A**
- 3. Fetch B**
- 4. ADD A and B**
- 5. Store the sum in C (Page fault!)**

Restart



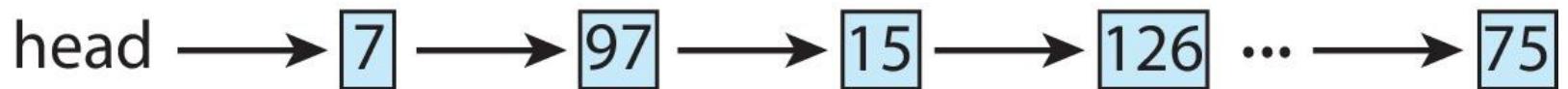
What happens if there is no free frame?

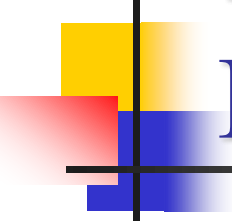


- ***Page replacement*** – find some page in memory, but not really in use, swap it out.
 - **algorithm**
 - **performance** – want an algorithm which will result in minimum number of page faults
- **Same page may be brought into memory several times.**



10.2.2 Free-Frame List





10.2.3 Performance of Demand Paging

- **Memory access time (*ma*)**

- about 10 to 200 ns.

ns (nanosecond) = 10^{-9}

- **Page-fault service time**

us (microsecond) = 10^{-6}

- total about 8 ms

ms (millisecond) = 10^{-3}

- **Three major components**

- Service the page-fault interrupt
- Read in the page
- Restart the process

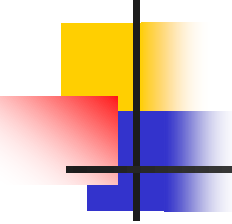
Effective Access Time (EAT) for a demand-paged memory

- **page fault rate (p)** $0 \leq p \leq 1.0$
 - if $p = 0$ no page faults
 - if $p = 1$, every reference is a fault

- **Effective Access Time (EAT)**

$$\text{EAT} = (1 - p) \times ma + p \times \text{page fault time}$$

- Directly proportional to the page-fault rate p



Demand Paging Example

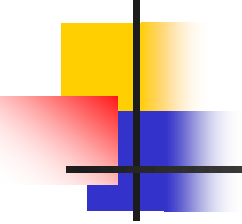
Assume

- Memory access time = 200 nanoseconds
- page fault service time = 8 milliseconds

$$\begin{aligned}\text{EAT} &= (1 - p) \times ma + p \times \text{Page fault time.} \\ &= (1 - p) \times 200 + p \times 8 \times 10^6 \\ &= 200 + 7,999,800 \times p \text{ (ns)}\end{aligned}$$

- For less than 10% degradation
 $\Rightarrow 220 > 200 + 7,999,800 \times p$
 $20 > 7,999,800 \times p$
 $p < 0.0000025$

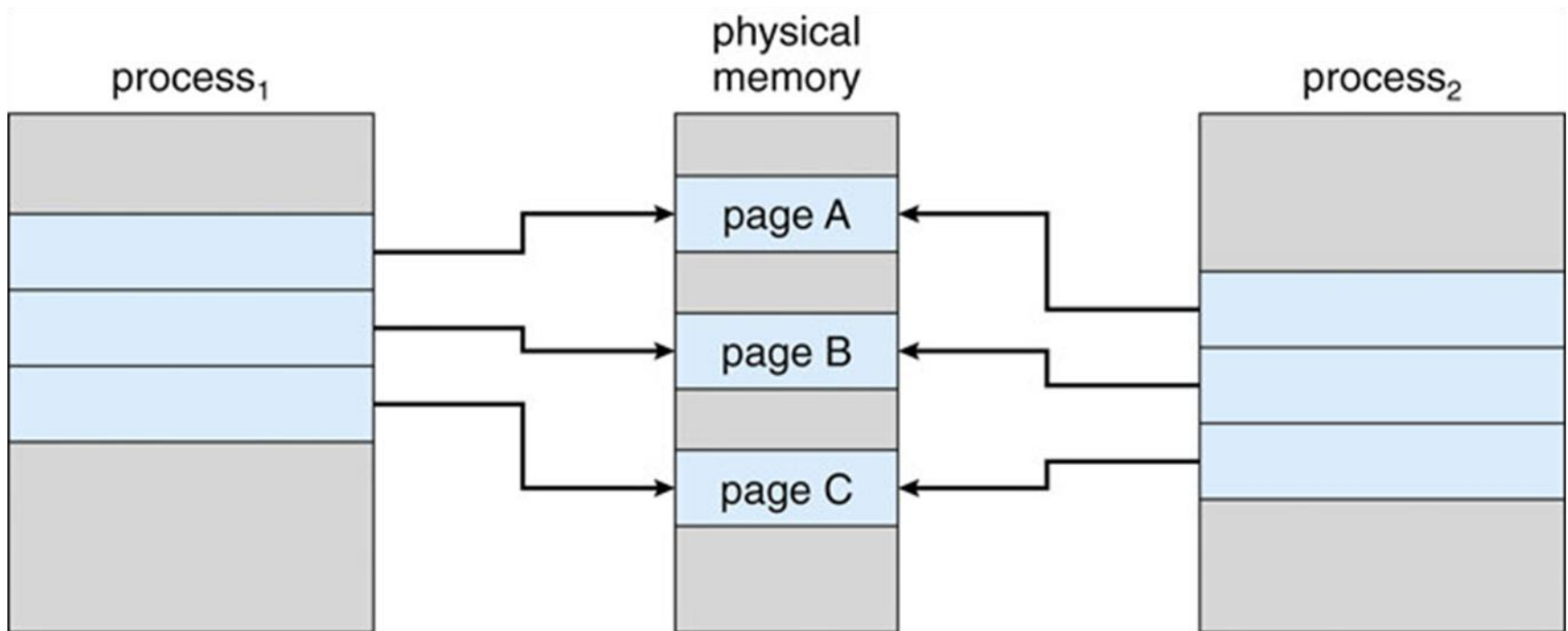




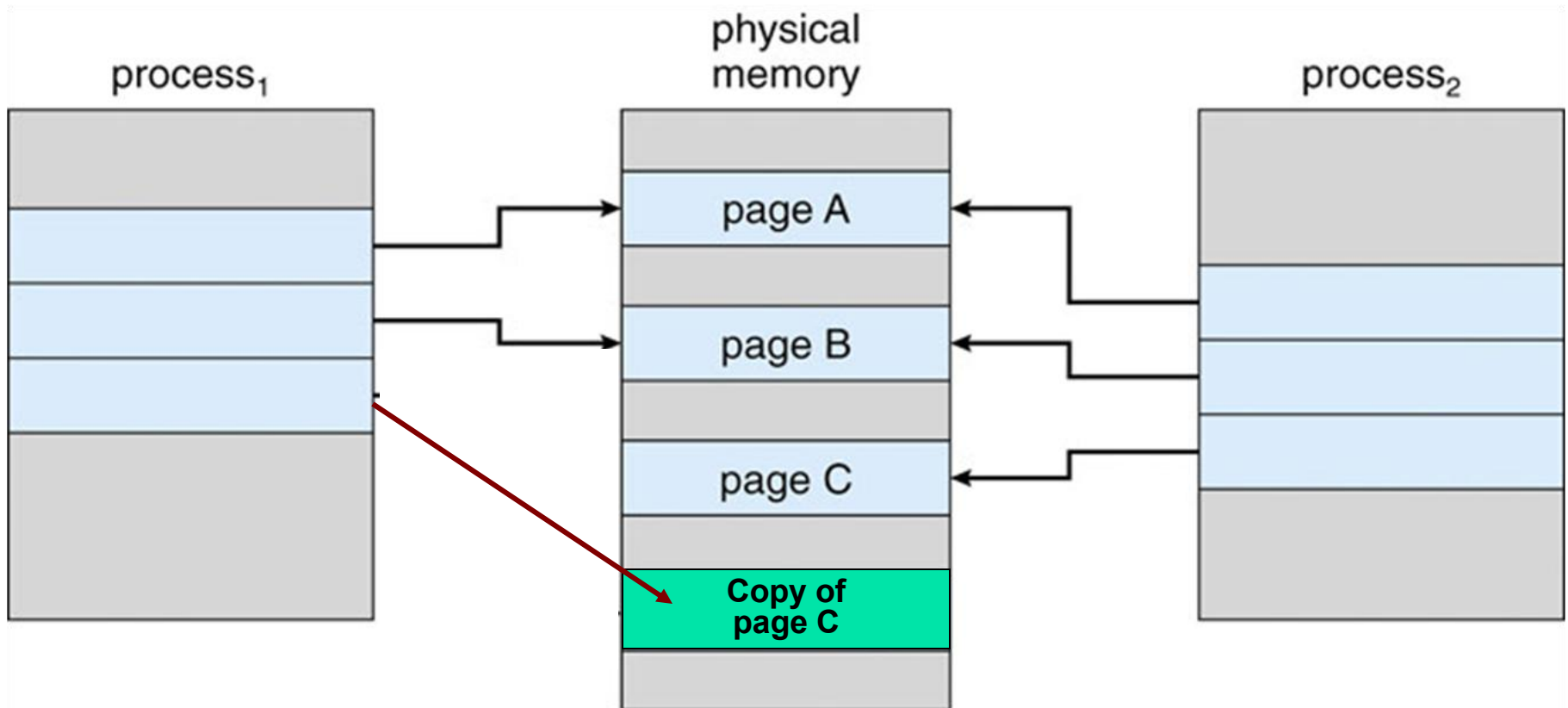
10.3 Copy-on-Write

- Traditionally **fork()** worked by creating a copy of the parent's address space for the child, duplicating the pages belonging to the parent.
- **Copy-on-Write (COW)** works by allowing both parent and child processes to initially *share* the same pages in memory.
If either process modifies a shared page, only then is the page copied.
- COW allows more efficient process creation as only modified pages are copied.
- A common technique used in Windows XP, Linux, Solaris.

Before process 1 modifies page C (Figure 10.7)



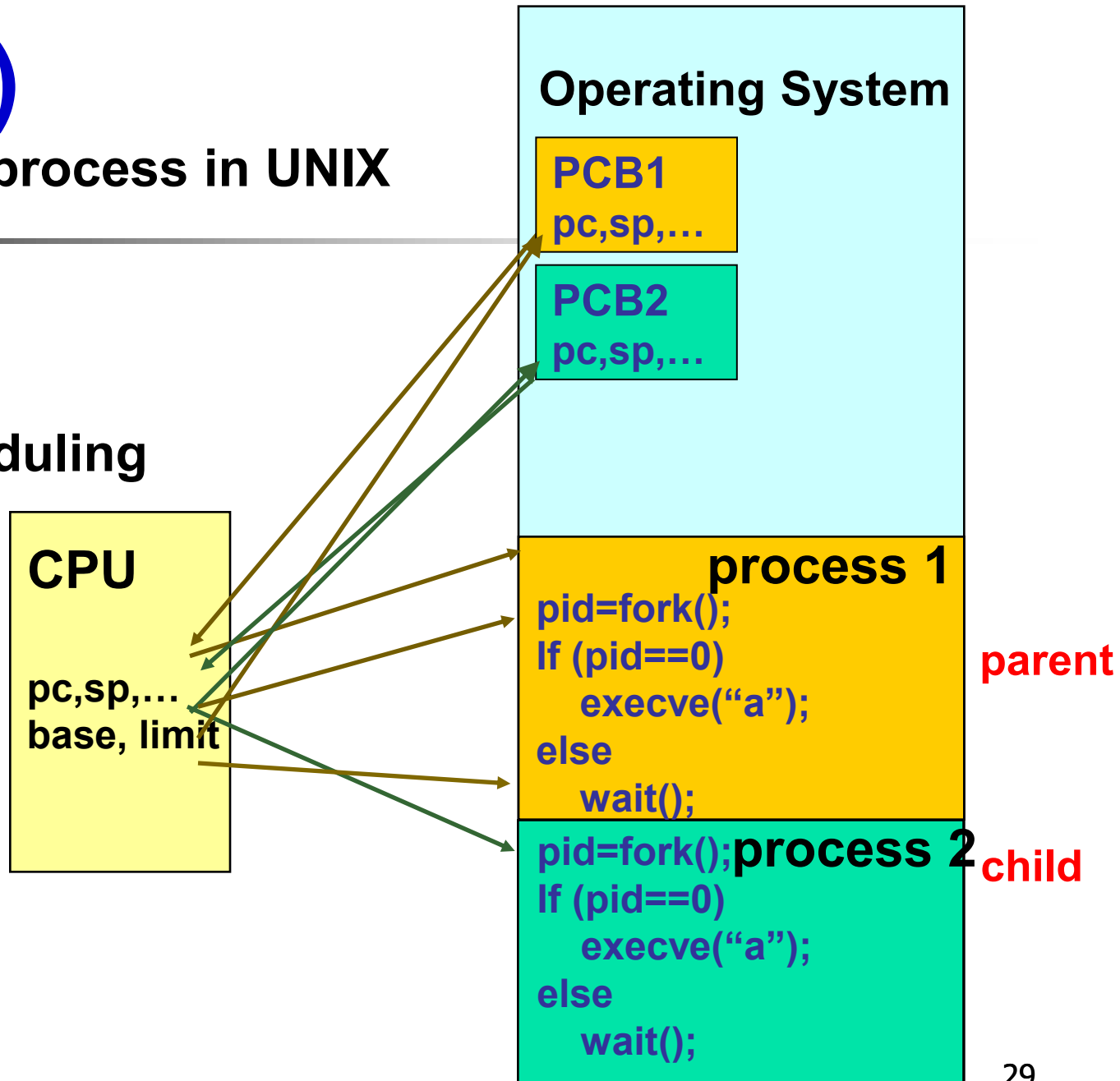
After process 1 modifies page C (Figure 10.8)



fork()

create a process in UNIX

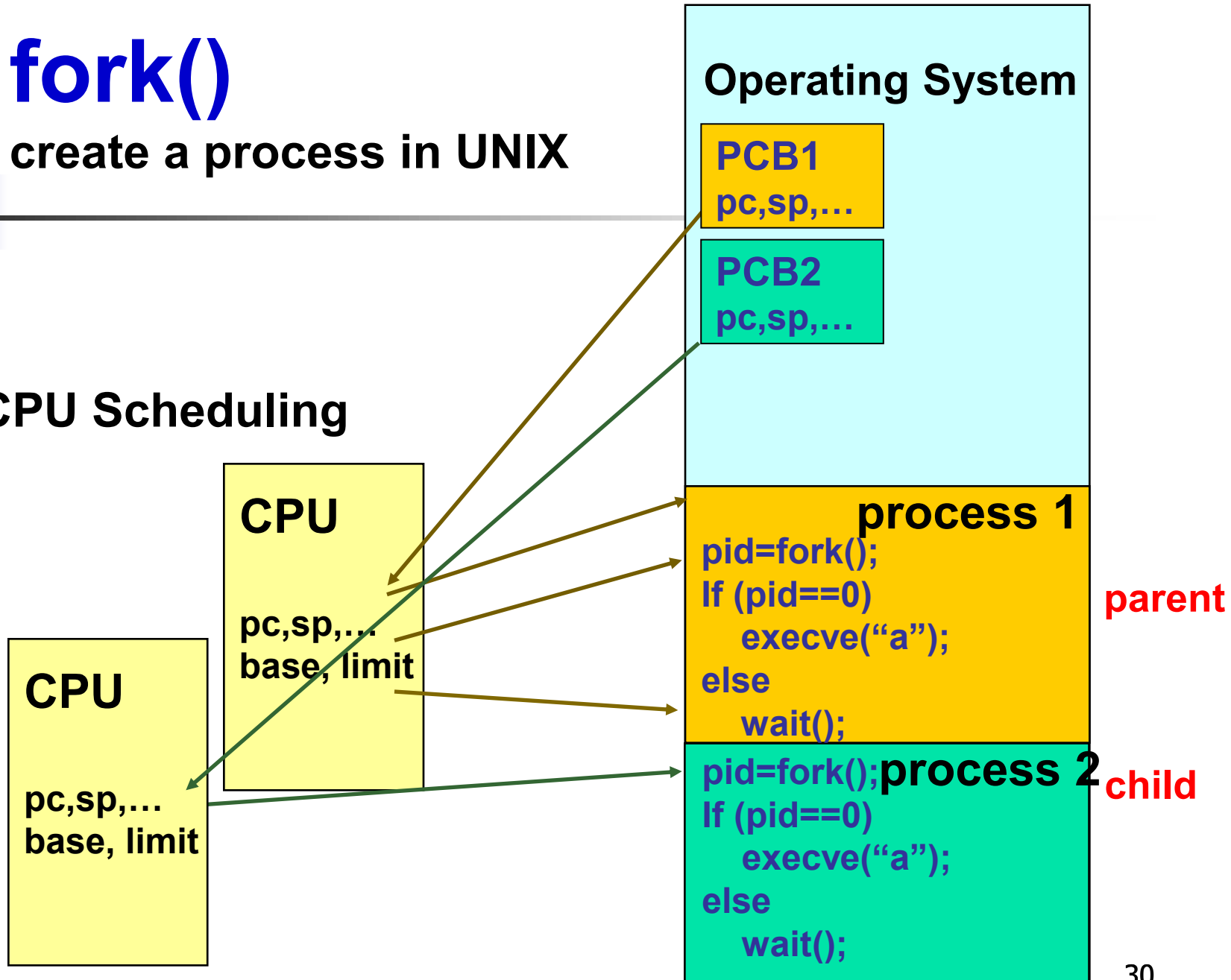
CPU Scheduling



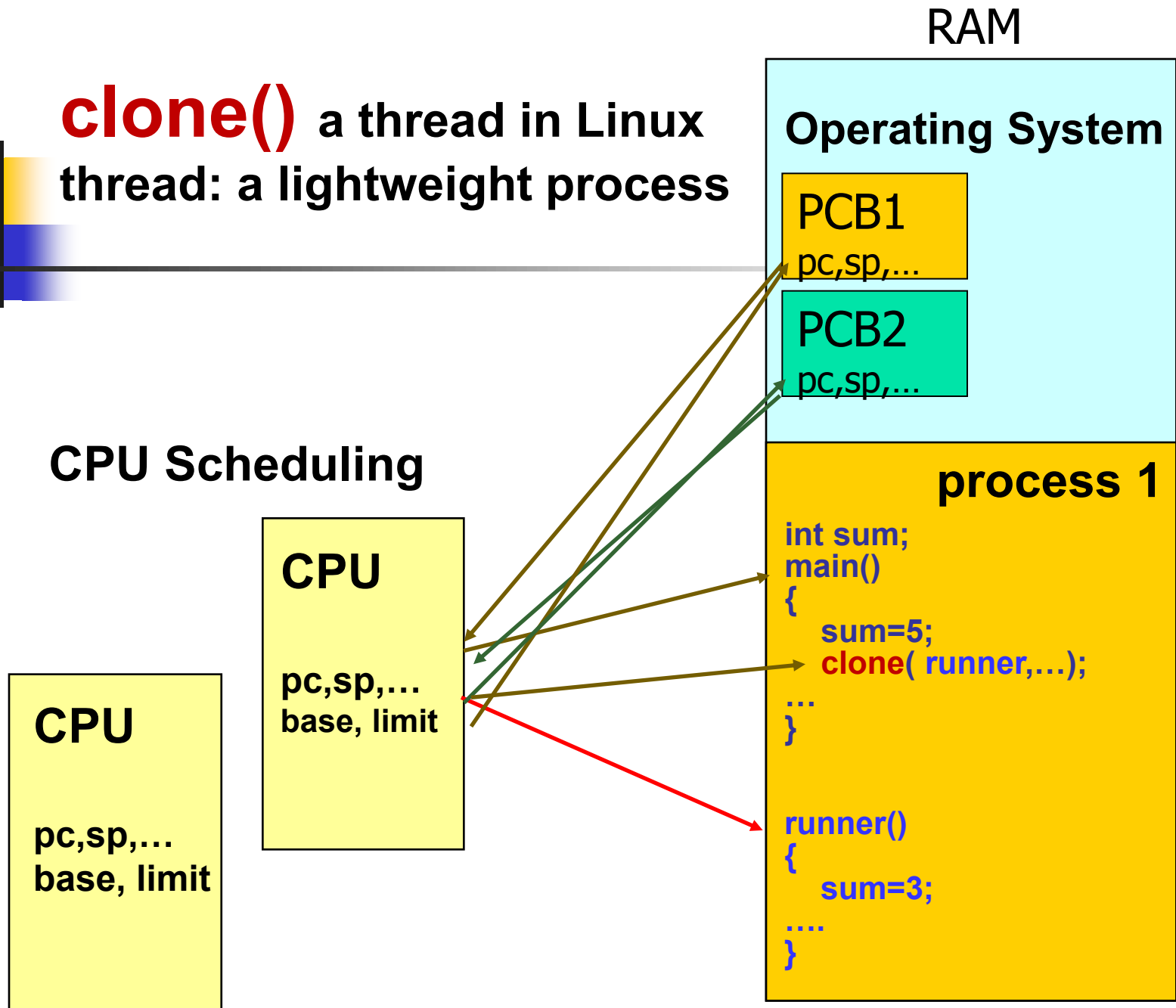
fork()

create a process in UNIX

CPU Scheduling

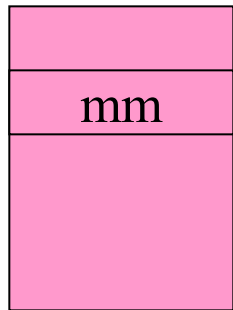


clone() a thread in Linux thread: a lightweight process

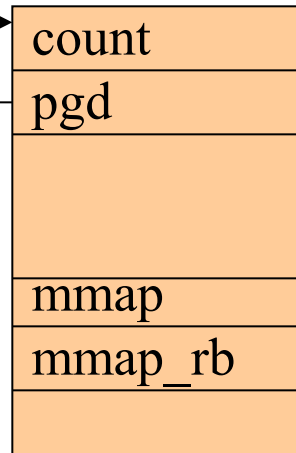


Linux Process's Virtual Memory

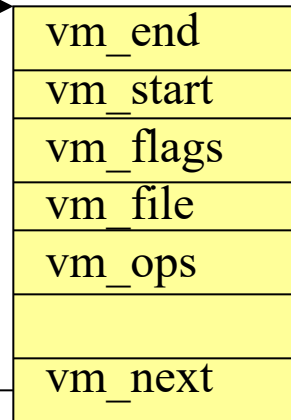
task_struct



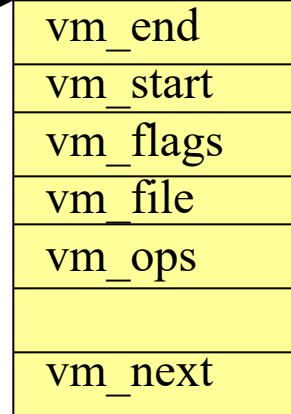
mm_struct



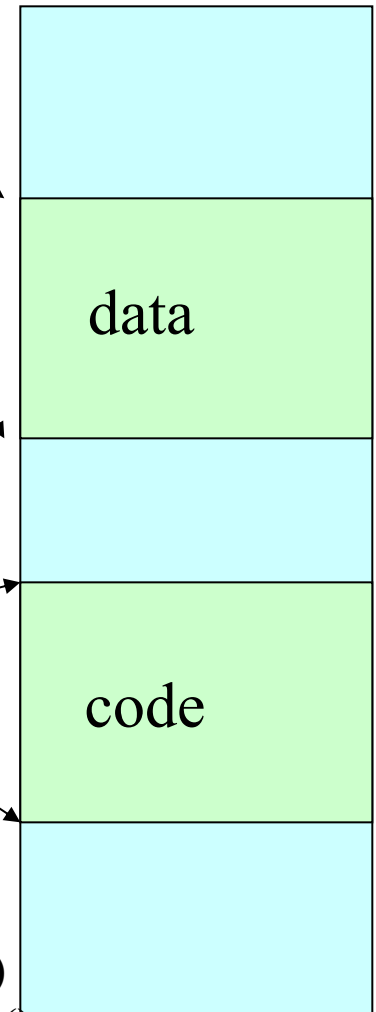
vm_area_struct



vm_area_struct

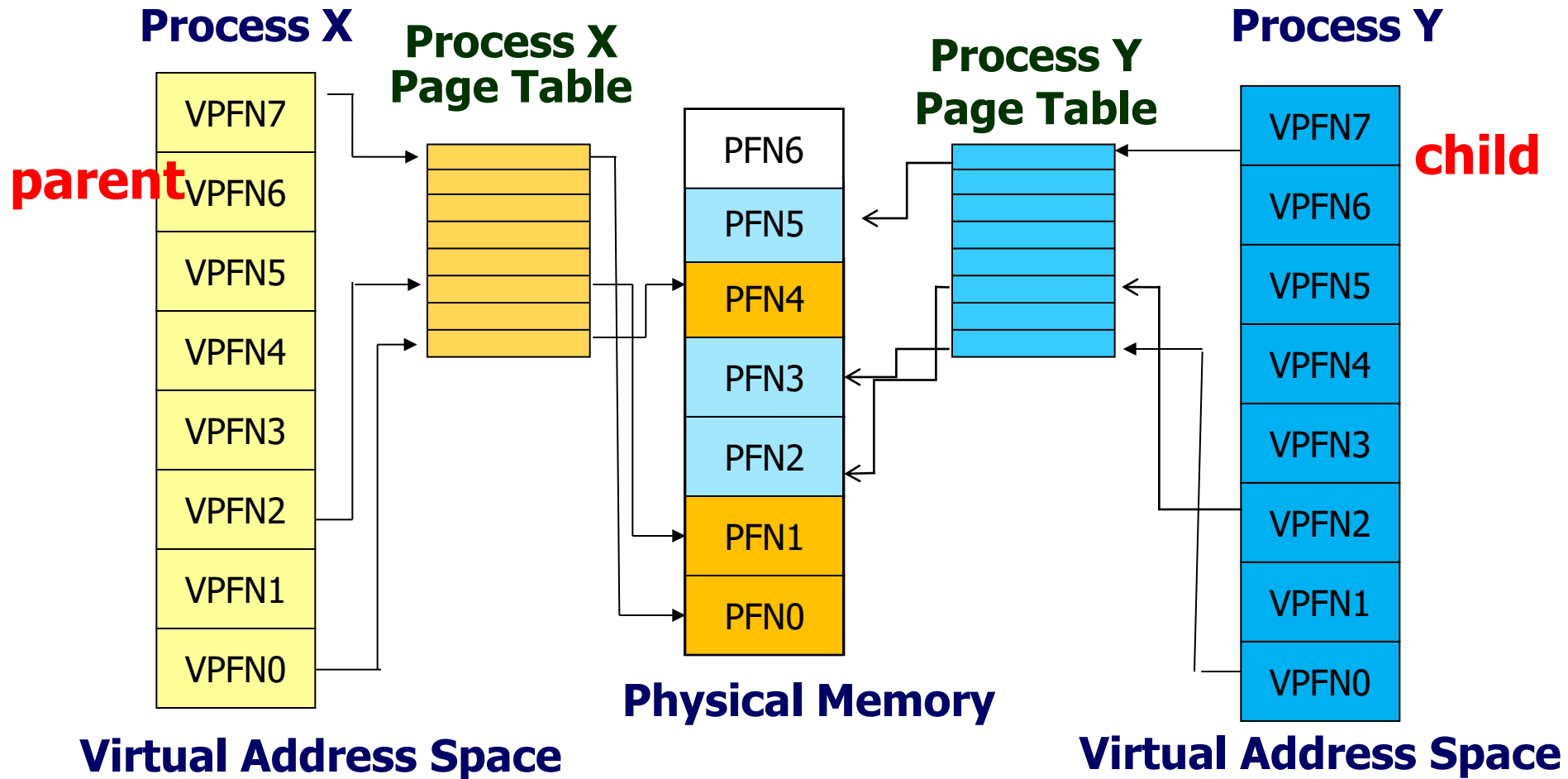


**Process's
Virtual Memory**



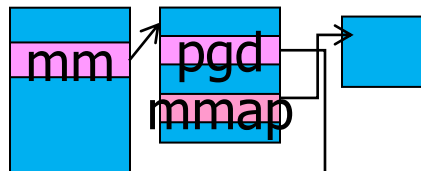
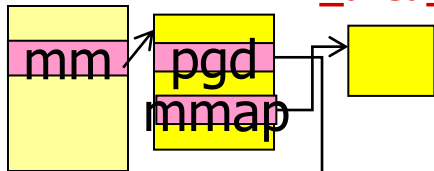
`open()`
`close()`
`nopage()`
`populate()`

Create a process



X
task_struct mm_struct
vm_area_struct

Y
task_struct mm_struct vm_area_struct



Create a process
Process Y

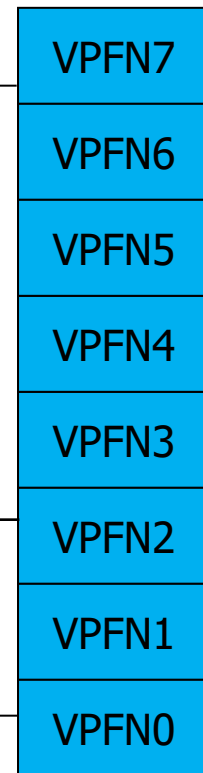
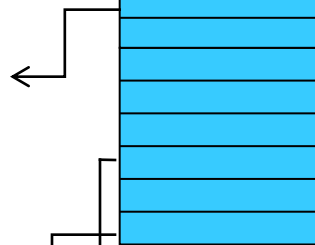
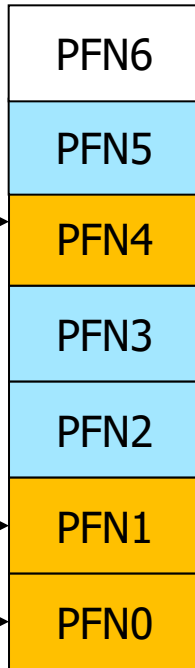
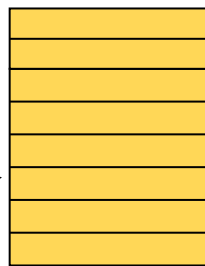
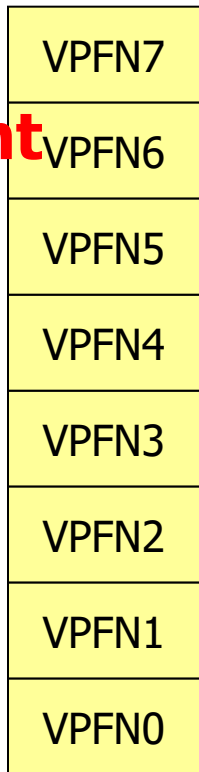
Process X

Process X Page Table

Process Y Page Table

child

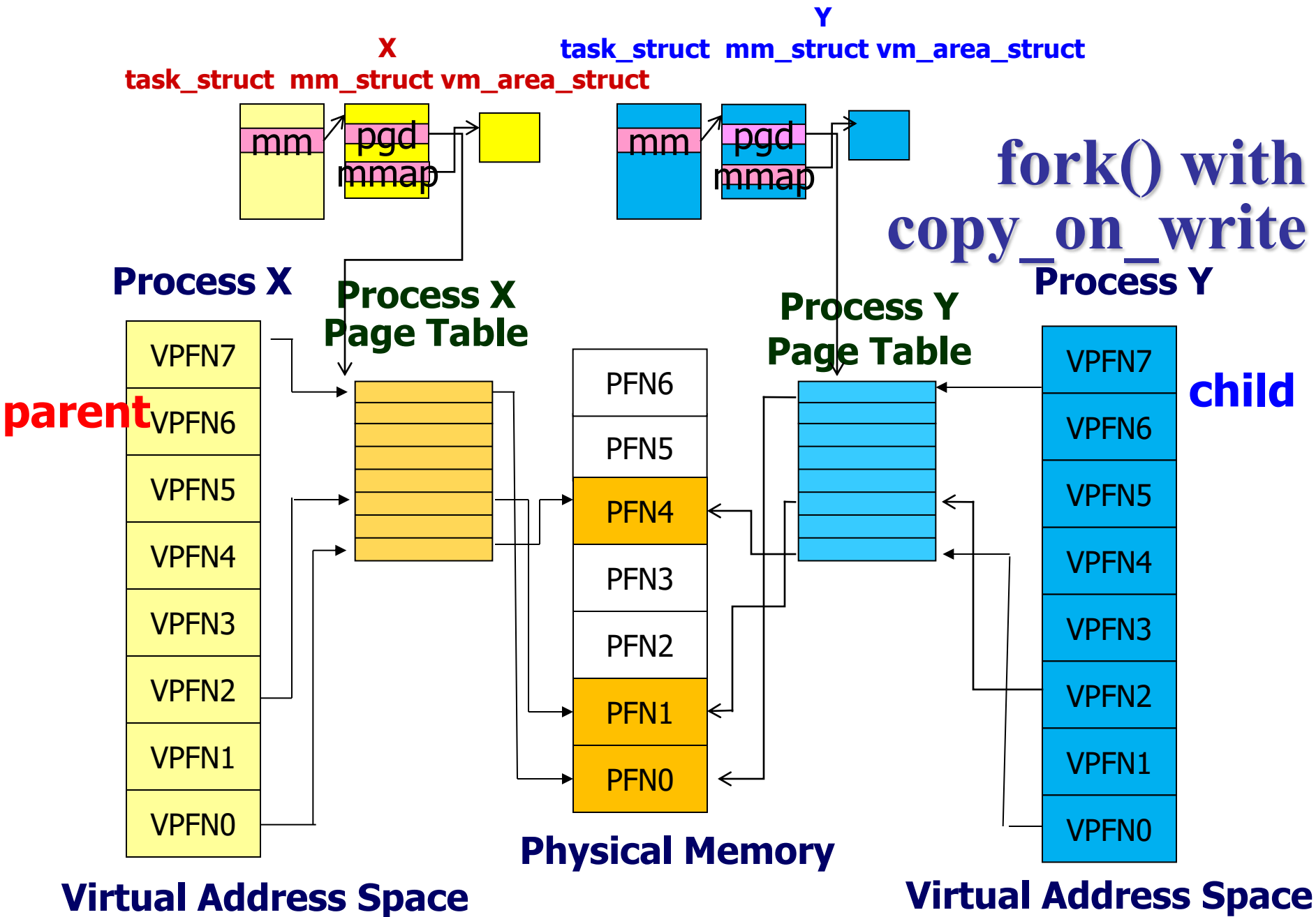
parent

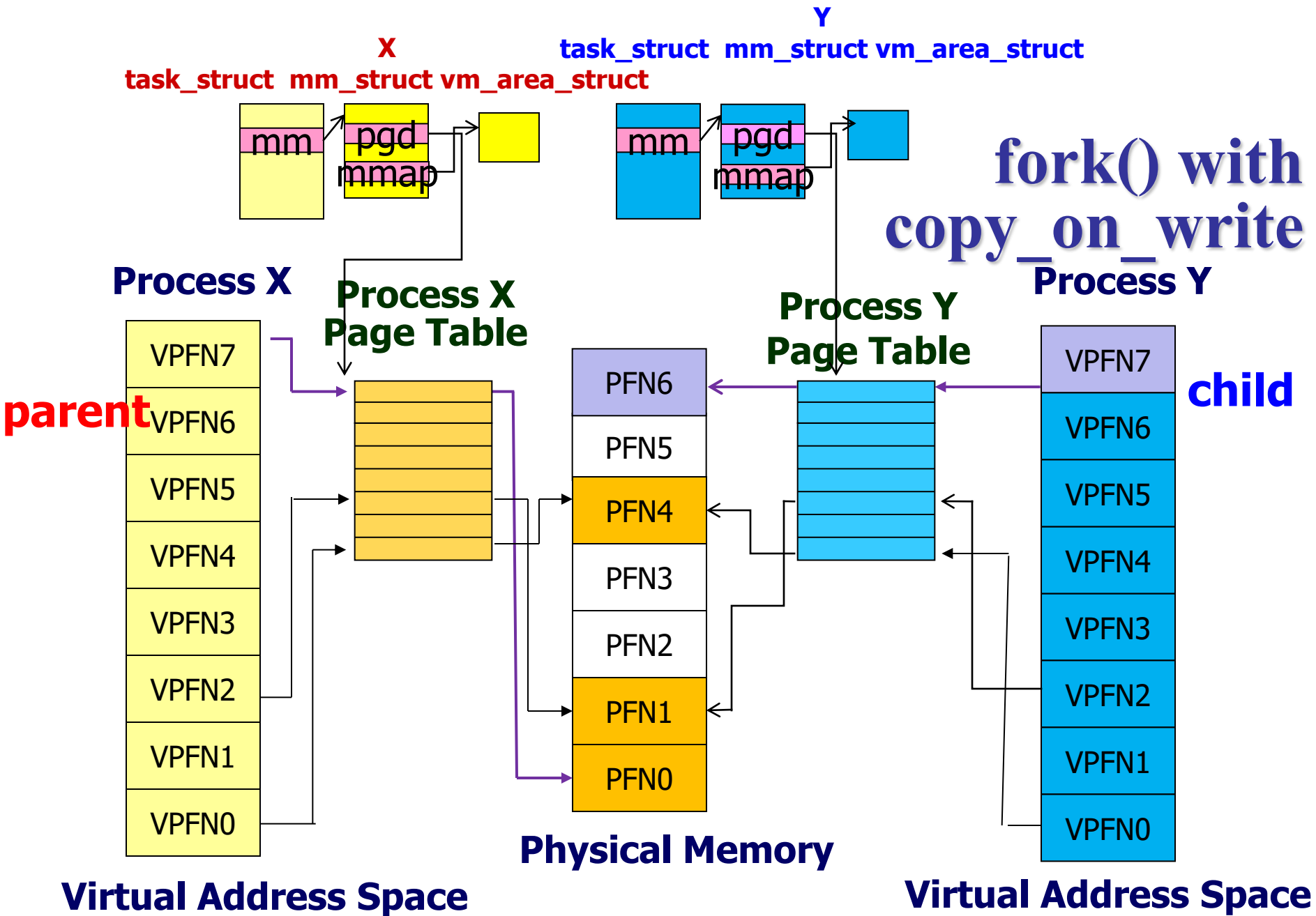


Virtual Address Space

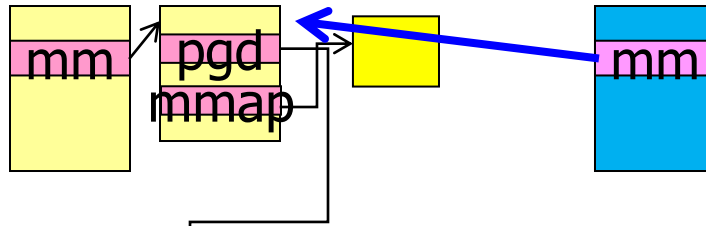
Physical Memory

Virtual Address Space





X
task_struct mm_struct vm_area_struct **Y task_struct**

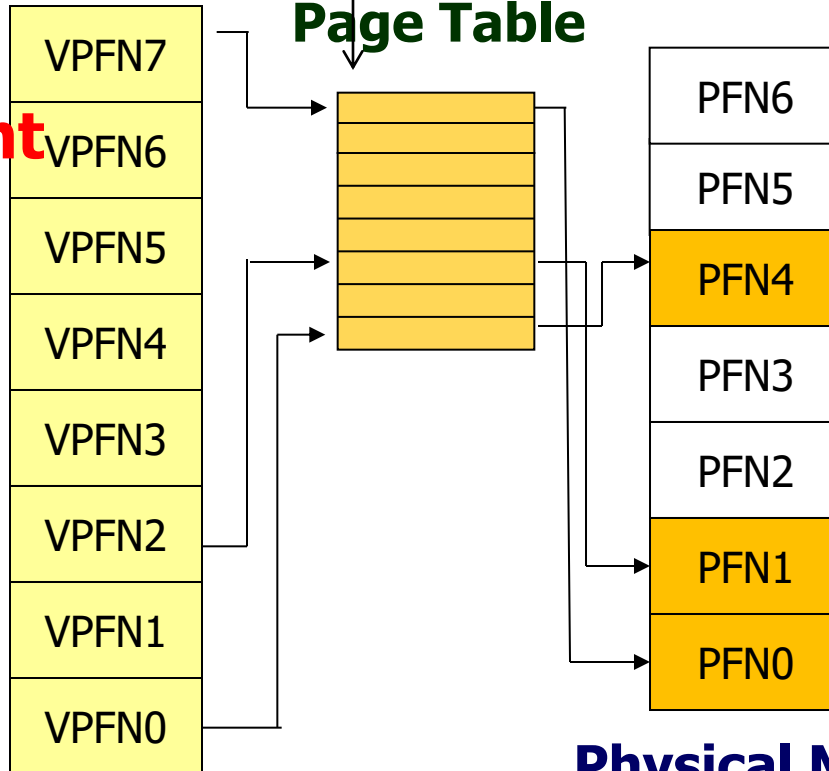


Create a light-weight process – clone()

Process X

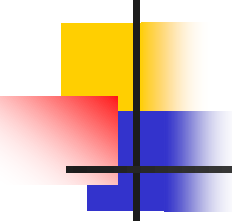
Process X Page Table

parent & child



Physical Memory

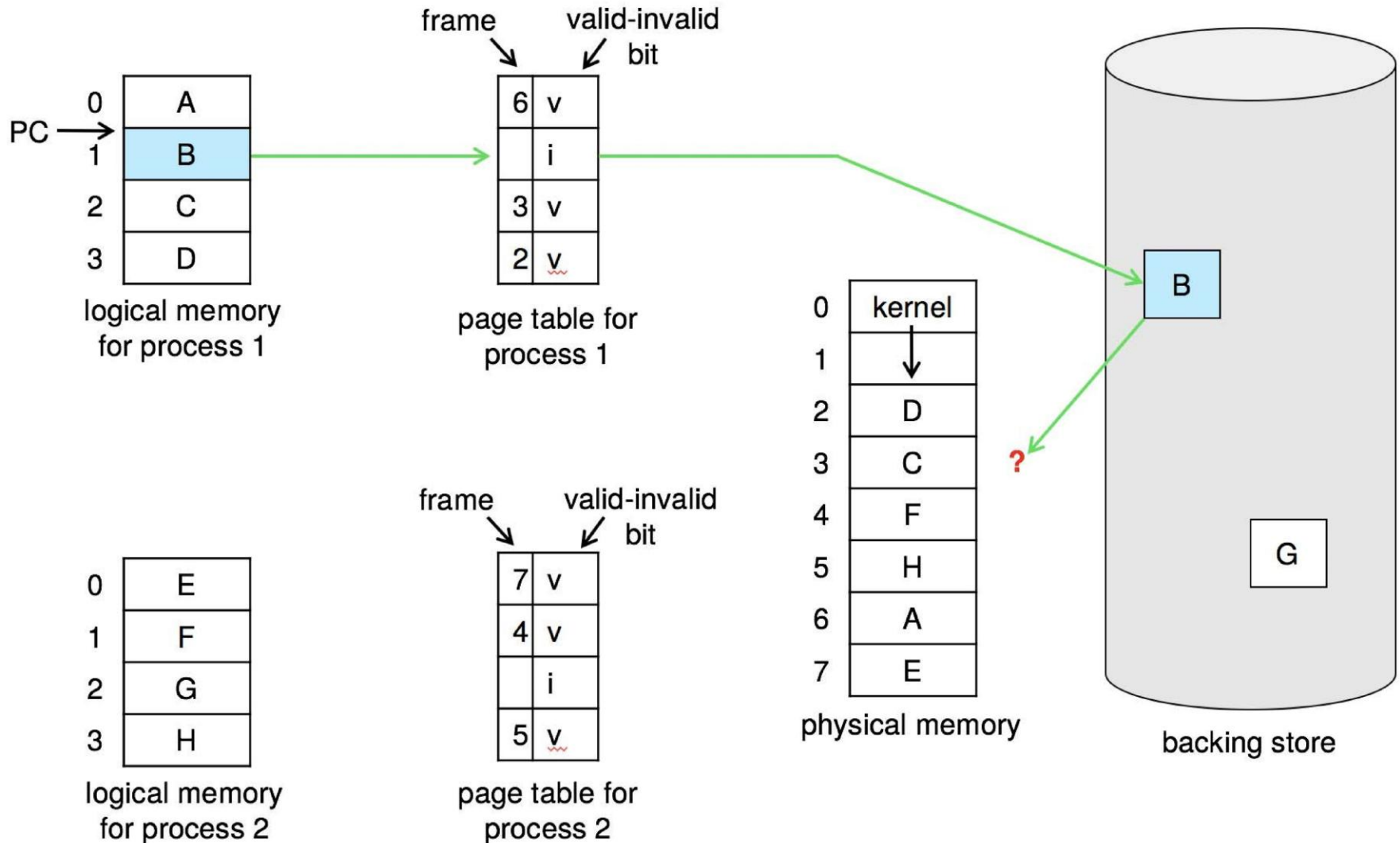
Virtual Address Space

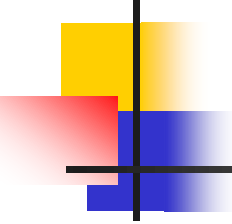


10.4 Page Replacement

- When there are no free-frames on free-frame list, OS could
 - terminate the user process
 - swap out a process, free all its frames, and reduce the level of multiprogramming
 - find a frame that is not currently being used and free it
- => *page replacement*

Need for Page Replacement (Fig. 10.9)

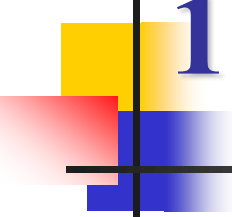




Page Replacement (Cont.)

- 10.4.1 Basic Page Replacement 
- 10.4.2 FIFO Page Replacement 
- 10.4.3 Optimal Page Replacement 
- 10.4.4 LRU Page Replacement 
- 10.4.5 LRU Approximation Page Replacement 
- 10.4.6 Counting-Based Page Replacement 
- 10.4.7 Page-Buffering Algorithms 
- 10.4.8 Applications and Page Replacement 

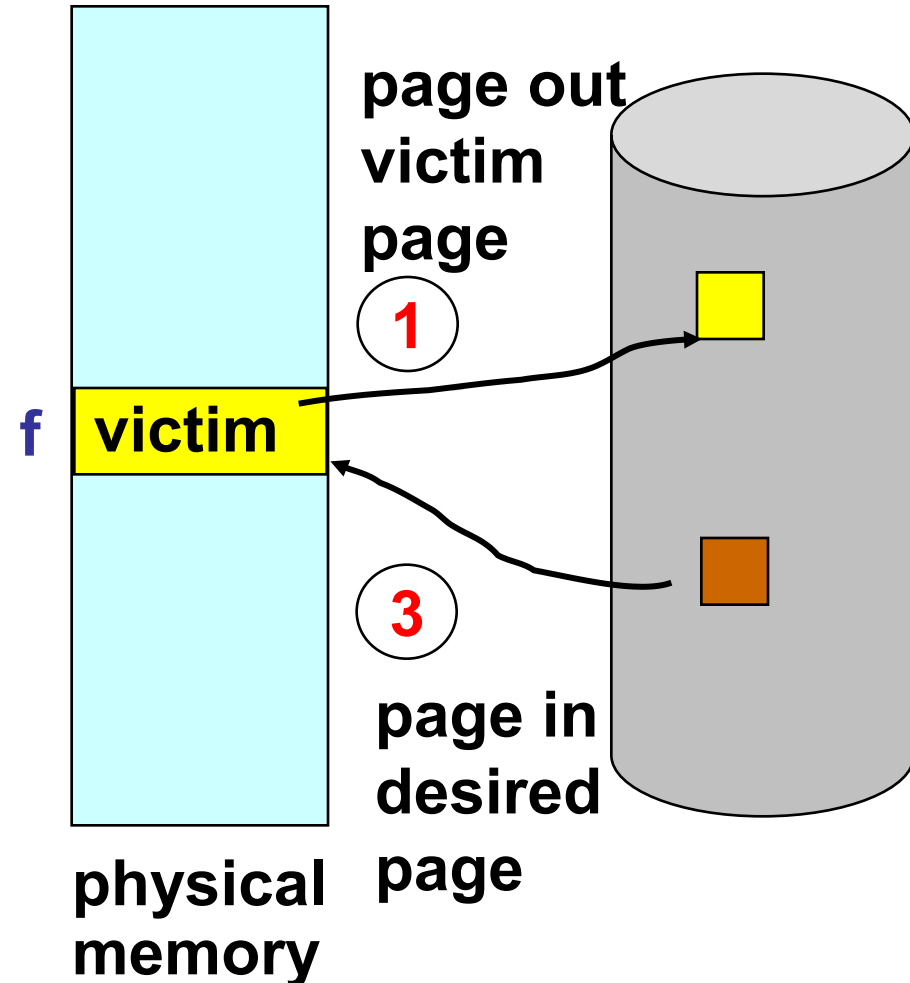
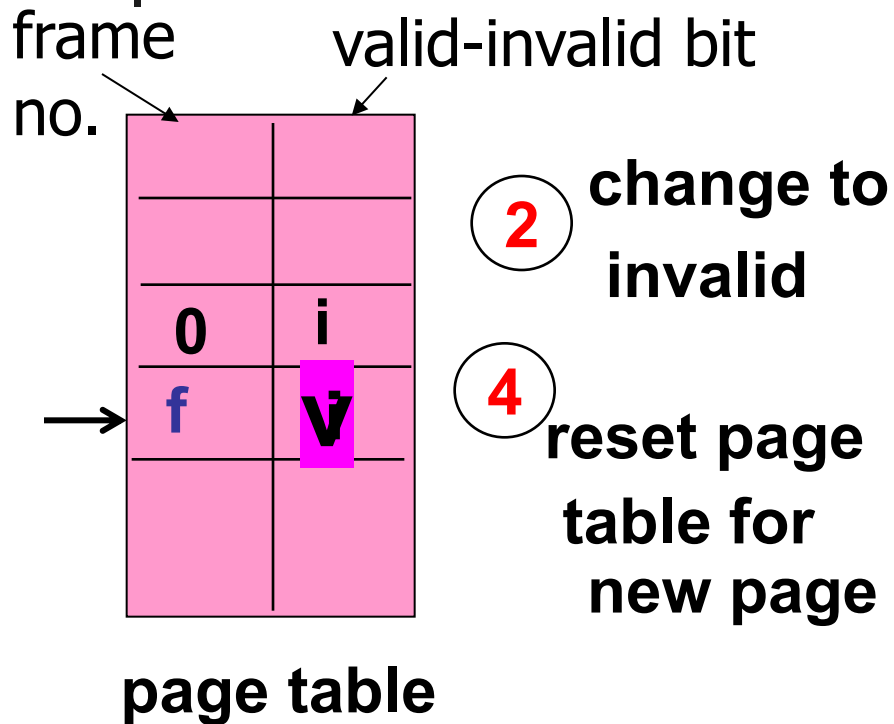


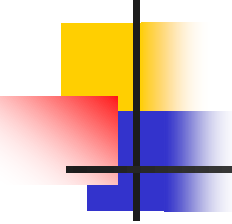


10.4.1 Basic Page Replacement

1. Find the location of the desired page on disk.
2. Find a free frame:
 - If there is a free frame, use it.
 - If there is no free frame, use a **page replacement algorithm** to select a *victim frame*.
 - write the victim frame to disk; change the page and frame tables accordingly
3. Read the desired page into the newly free frame; change the page and frame tables.
4. Restart the user process.

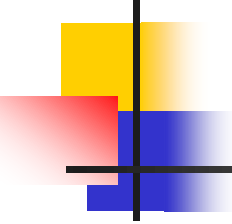
Page Replacement (Fig. 10.10)





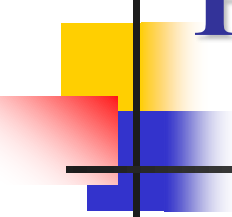
Page Replacement (Cont.)

- To reduce the time required to service a page fault, to free a frame in page replacement:
 - a **read-only** page may be discarded when desired
 - only **modified (dirty)** pages must be written to swap space
 - examine *modify (dirty) bit*
- Page-fault service routine should include page replacement.
- Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory.



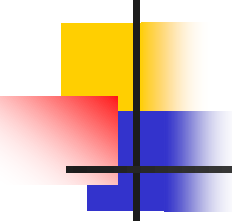
Page Replacement (Cont.)

- To implement demand paging, we need
 - **A frame-allocation algorithm**
 - decides how many frames to allocate to each process
 - **A page-replacement algorithm**
 - selects the frames that are to be replaced



Page-Replacement Algorithms

- Want the one with the **lowest page-fault rate**
- To evaluate an algorithm
 - run it on a particular string of memory references
 - compute the number of page faults
- ***reference string***
 - the string of memory references
 - consider only the page number rather than entire address
 - page reference immediately follows the same page does not count

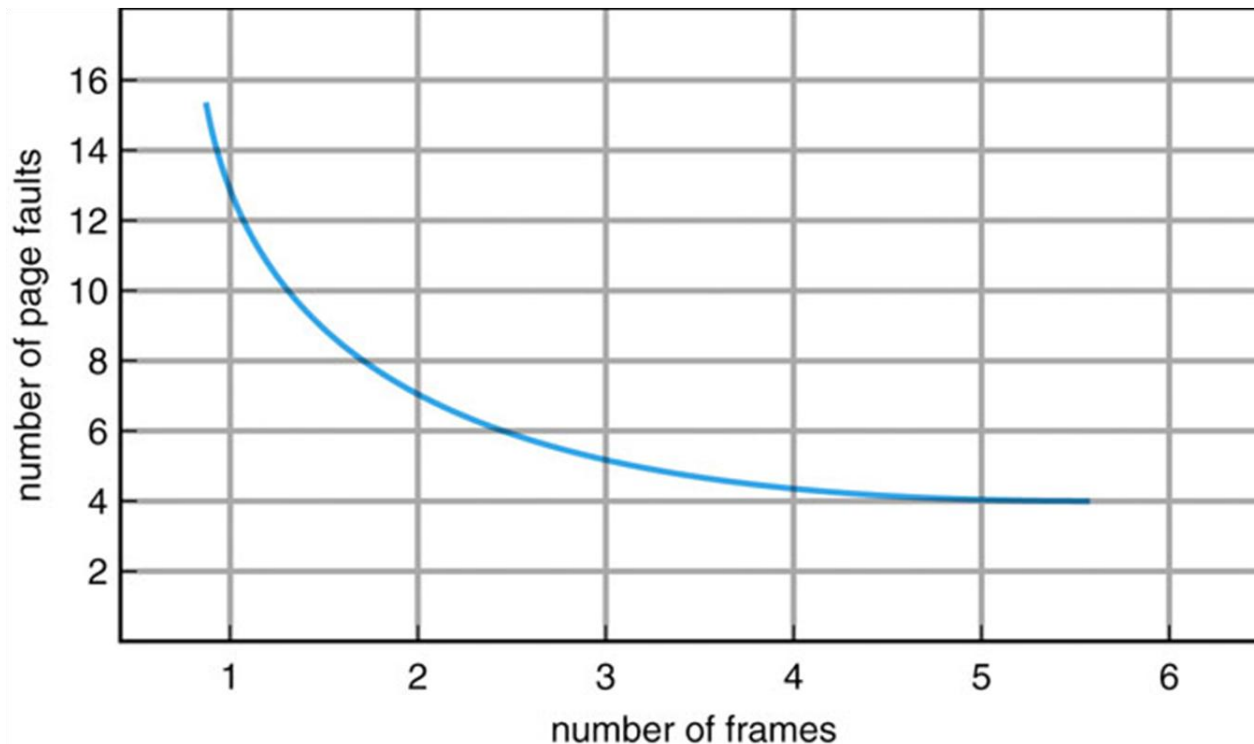


Reference String Example

- **Address sequence recorded:**
0100, 0432, 0101, 0612, 0102, 0103, 0104,
0611, 0102, 0103, 0601, 0102, 0104, 0609,
0100, 0105
- **If page size = 100 $\Rightarrow$ reference string =**
1, 4, 1, 6, 1, 6, 1, 6, 1, 6, 1

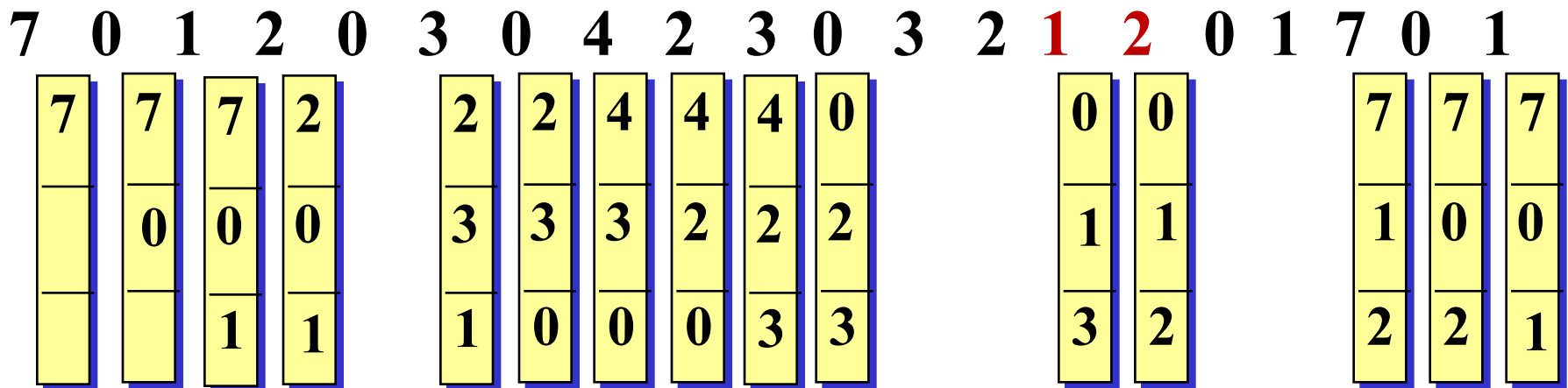
Page Faults vs. Number of Frames

- Illustration reference string (with three frames)
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
- Graph of Page Faults vs. The Number of Frames (Fig. 10.11)



10.4.2 FIFO Page Replacement

- Associates with each page the time when it was brought into memory.
- The oldest page is chosen to be replaced
- Can create a FIFO queue to hold all pages in memory.



page-faults

Total 15 page faults

異常

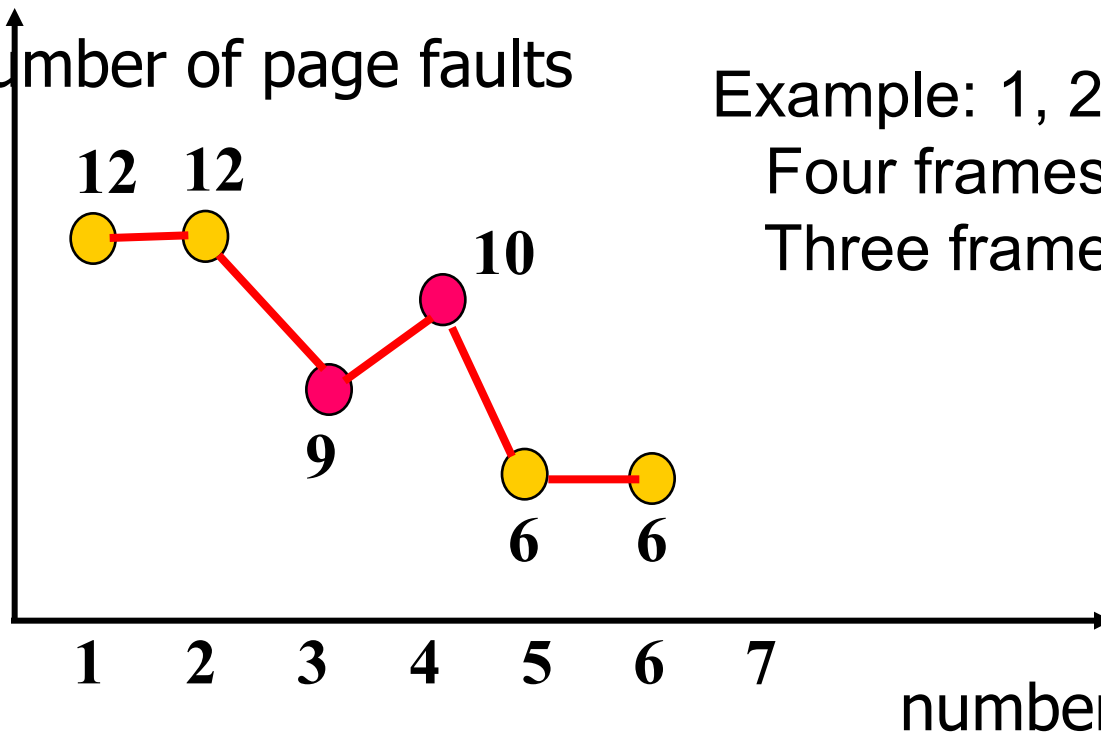
Belady's Anomaly

- **Belady's anomaly:**

allocated frames $\uparrow$ page-fault rate $\uparrow$

number of page faults

Example: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
Four frames $\Rightarrow$ 10 page faults but
Three frames $\Rightarrow$ 9 page faults



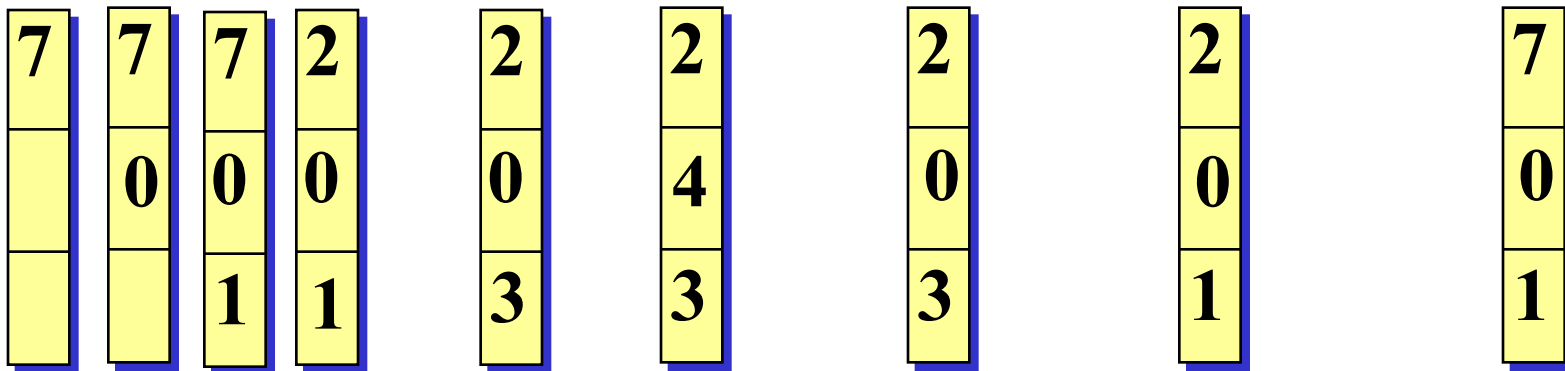
10.4.3 Optimal Page Replacement (OPT; MIN)

將來最久沒用到的page

將來最久才會用到的page

- **Lowest page-fault rate** of all algorithms
- Replace the page that **will not be used** for the longest period of time.

7 0 1 2 0 **3** 0 4 2 3 **0** 3 2 1 2 0 1 7 0 1



Total 9 page faults

- Difficult to implement because it requires future knowledge of the reference string
 - used mainly for comparison studies.



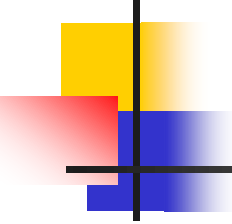
10.4.4 LRU Page Replacement (Least Recently Used)

- Looking backward in time, rather than forward.
- Replace the page that **has not been used for the longest period time** 已經最久沒用到的page

7 0 1 2 0 **3** 0 4 2 3 0 3 2 **1** 2 0 1 7 0 1

7	7	7	2	2	4	4	4	0	1	1	1
	0	0	0	0	0	0	3	3	3	0	0
		1	1	3	3	2	2	2	2	2	7

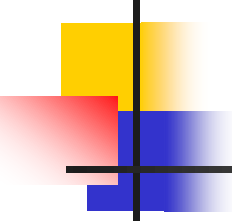
Total 12 page faults



LRU Page Replacement (Cont.)

- LRU page replacement algorithm may require substantial hardware assistance
- Two implementations are feasible:
 - Counters 
 - Stacks 
- Need hardware assistance since updating of the clock fields or stack must be done for every memory reference.

LRU Implementation: Counters



- Associate with each page-table entry a time-of-use field
- Add to the CPU a logical clock or counter
 - The clock is incremented for every memory reference
- Whenever a reference to a page is made, copy the clock value to the time-of-use field of that page
- Replace the page with the smallest time value (needs searching)



LRU Implementation: Stack

- Keep a stack of page numbers in a doubly linked list:

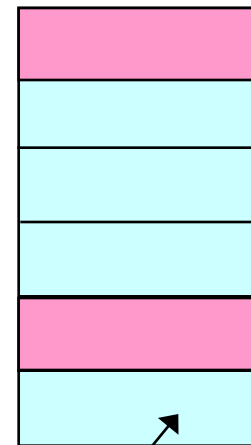
- Page referenced:

- move it to the top

- No search for replacement

MRU page

LRU page

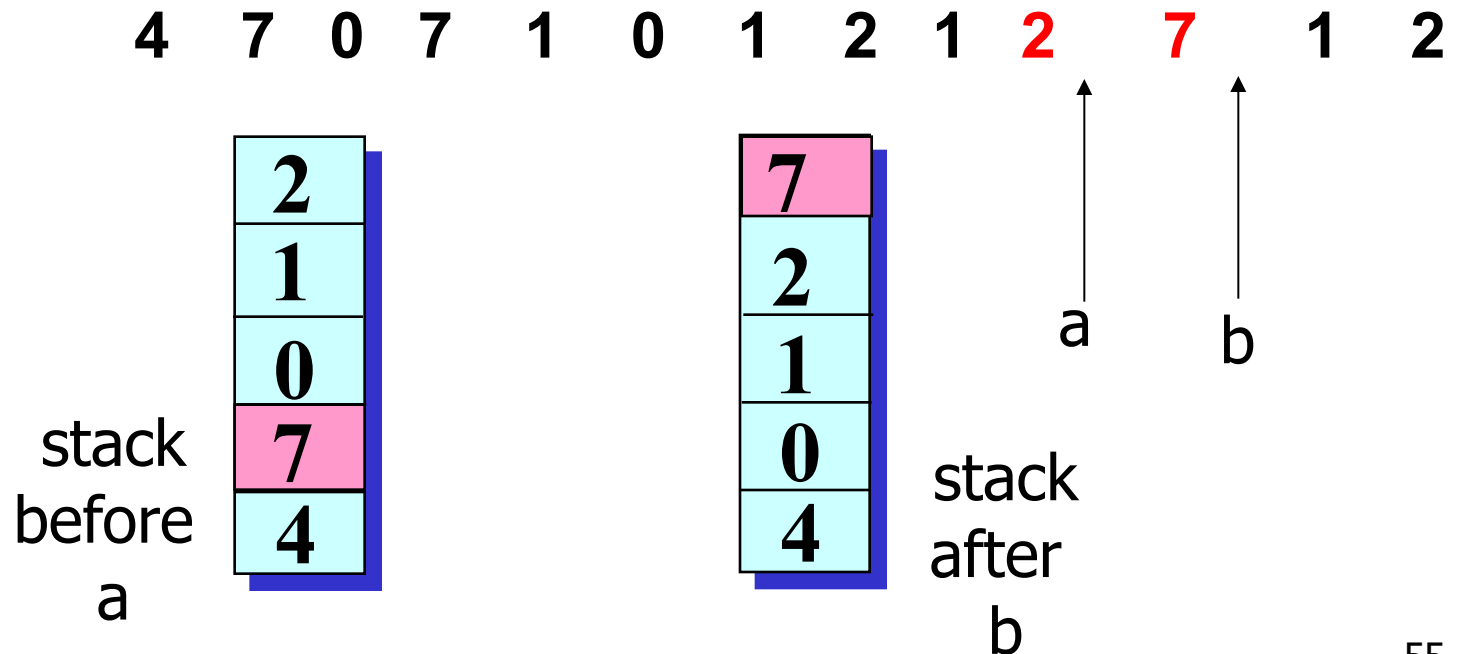


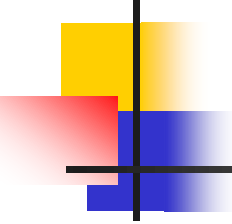
move to top
when it is
referenced

LRU Implementation: Stack

(Cont.)

- Use a stack to record the most recent page references (Fig. 10.16):

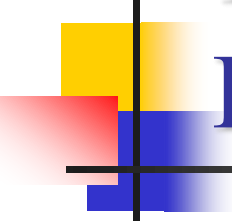




Stack Algorithms

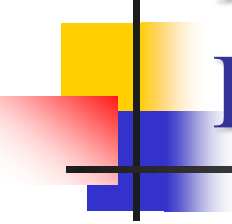
- There is a class of page-replacement algorithms, called ***stack algorithms***, such that
 - the set of pages in memory for ***n*** frames is always a subset of the set of pages that would be in memory with ***n + 1*** frames
- Stack algorithm never exhibits Belady's anomaly
 - both OPT and LRU are stack algorithms





10.4.5 LRU-Approximation Page Replacement

- Few systems provide sufficient hardware support for LRU page-replacement
- Many systems provide some help: *reference bits*
- **Reference bit**
 - Associated with each page table entry
 - Set by hardware whenever that page is referenced
- **Approximate LRU replacement**
 - Replace the page whose reference bit is 0
 - However, we do not know the order => so, this is a approximation to LRU



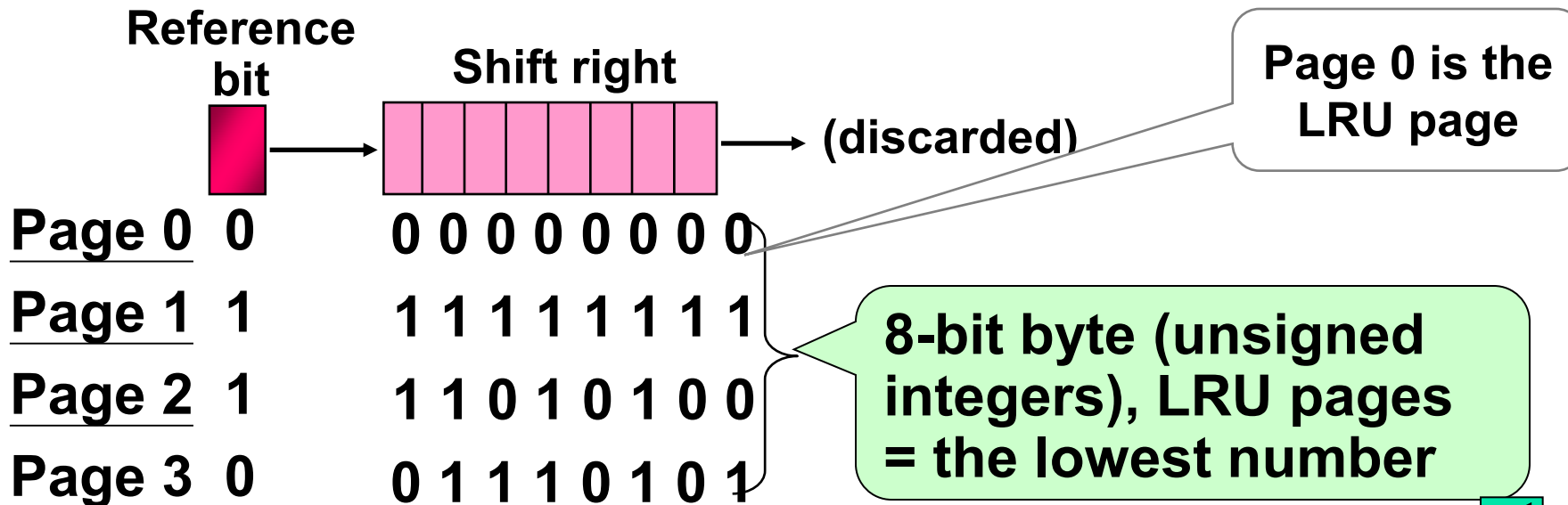
LRU-Approximation Page Replacement (Cont.)

- **10.4.5.1 Additional-Reference-Bits Algorithm** 
- **10.4.5.2 Second-Chance Algorithm** 
- **10.4.5.3 Enhanced Second-Chance Algorithm**



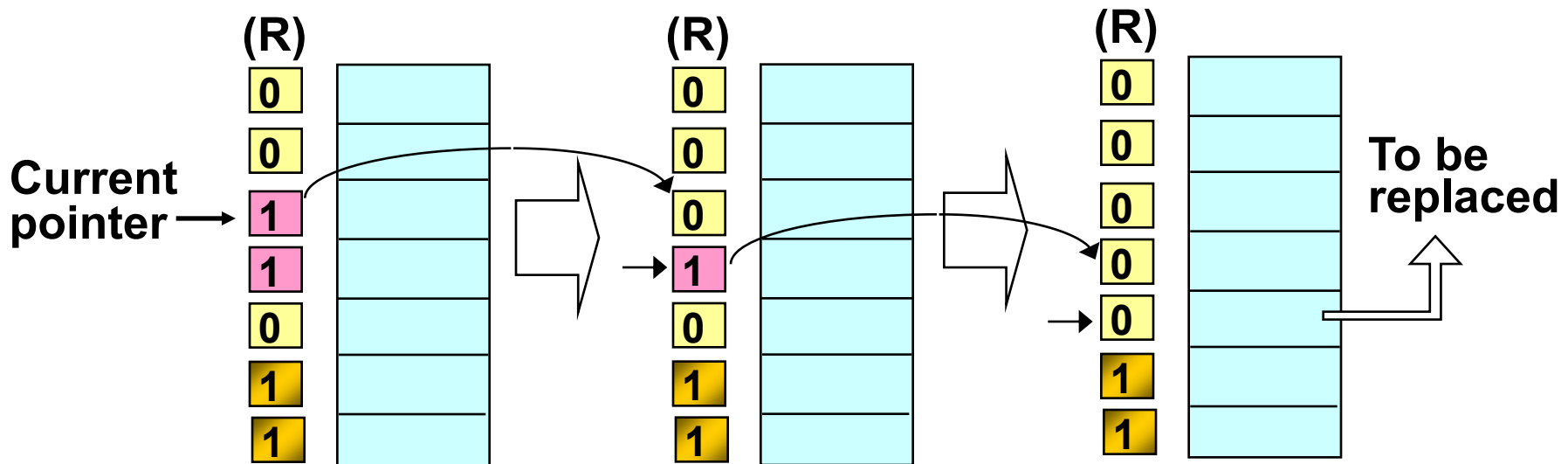
10.4.5.1 Additional-Reference-Bits Algorithm

- **Record the reference bits at regular intervals**
 - keeps in memory an 8-bit byte (as 8-time history) for each page in a table



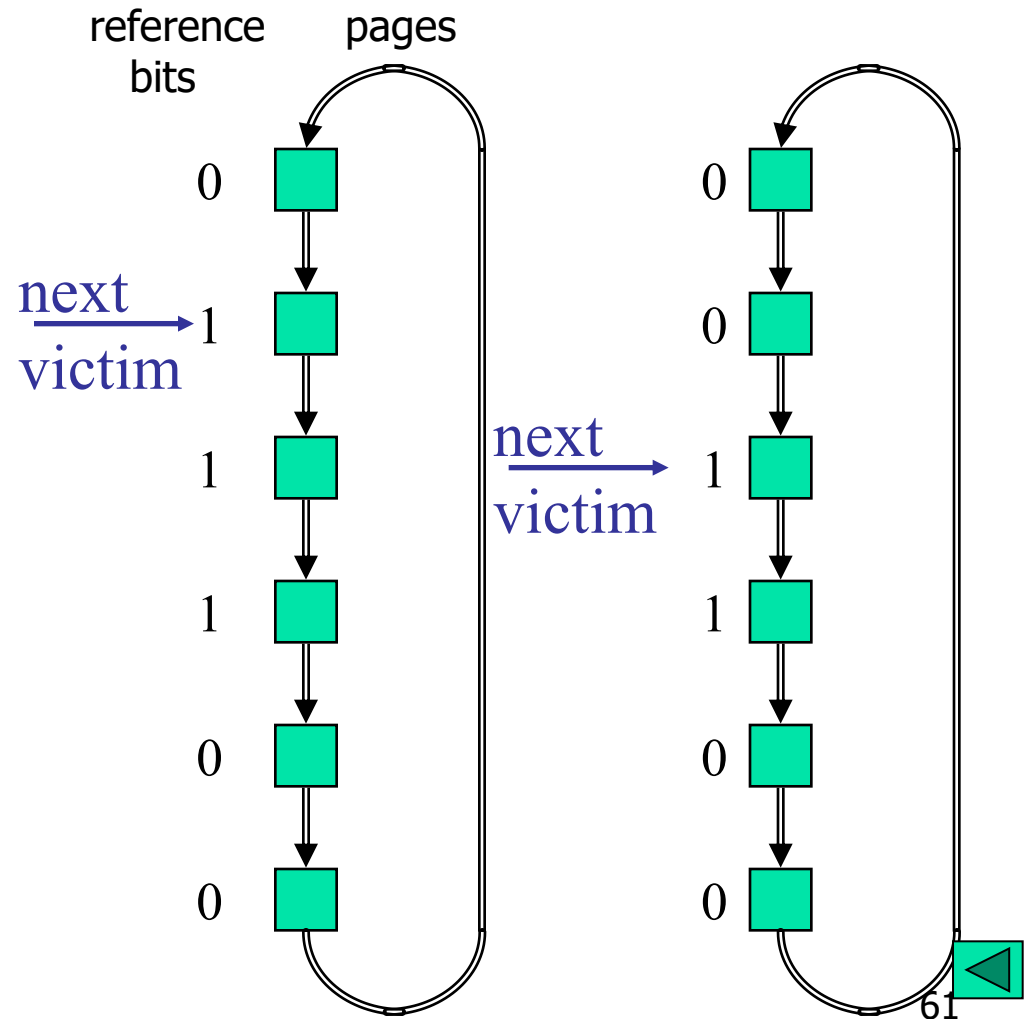
10.4.5.2 Second-Chance (Clock) Algorithm

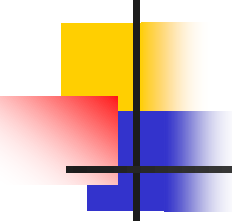
- Basically a **FIFO** replacement algorithm, however
 - if the reference bit is 1, we give that page a **second chance** (clear its reference bit and reset its arrival time to the current time)



Second-Chance (Clock) Algorithm (Cont.)

- Can be implemented as a *circular queue*
- Need **reference bit**.
- **Clock replacement.**





10.4.5.3 Enhanced Second-Chance Algorithm

- Consider both the **reference bit** and the **modify bit**
- (**reference bit**, **modify bit**) pair defines 4 classes of pages
 - **(0, 0): neither recently used nor modified**
 - Best page to replace
 - **(0, 1): not recently used but modified**
 - Not quite as good, because the page need to be written out
 - **(1, 0): recently used but clean**
 - Probably will be used again soon
 - **(1, 1): recently used and modified**
 - Probably will be used again, must write out before replacing it

Enhanced Second-Chance Algorithm (Cont.)

- Use the same scheme as the clock algorithm, but examine the class to which that page belongs.
- First search all pages in the lowest class, then the 2nd lowest class, ... etc.
- Replace the first page encountered in the lowest nonempty class
 - may scan the circular queue several times

10.4.6 Counting-Based Page Replacement

- Keep a counter of the number of references that have been made to each page
- **LFU Algorithm (Least Frequently Used)**
 - Replace the page with the smallest count
 - ∴ an actively used page should have a large reference count
- **MFU Algorithm (Most Frequently Used)**
 - The page with the smallest count was probably just brought in and has yet to be used



10.4.7 Page Buffering Algorithms

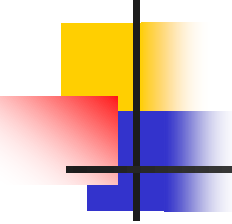
- **Systems commonly keeps a pool of free frames**
 - the desired page is read before victim is written out
 - allows the process to restart as soon as possible
- **Maintain a list of modified pages**
 - whenever paging device is idle, a modified page is written to the disk (and its modify bit is reset)
- **Keep a pool of free frames but to remember which page was in each frame**
 - when a page fault occurs, we first check whether the desired page is in the free-frame pool
 - Used in VAX/VMS system, with a FIFO replacement



10.4.8 Applications and Page Replacement

- Some OS provides applications with their own memory management and I/O buffering.
 - give special programs the ability to use a disk partition as a large sequential array of logical blocks, without any file-system data structures.
 - This array is sometimes called the **raw disk**, and I/O to this array is termed **raw I/O**

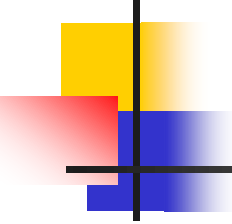




10.5 Allocation of Frames

- How do we allocate the fixed amount of free memory among the various processes?
- 10.5.1 Minimum Number of Frames 
- 10.5.2 Allocation Algorithms 
- 10.5.3 Global vs. Local Allocation 
- 10.5.4 Non-Uniform Memory Access 

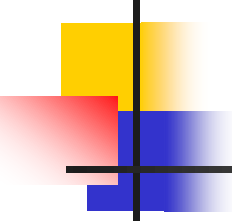




10.5.1 Minimum Number of Frames

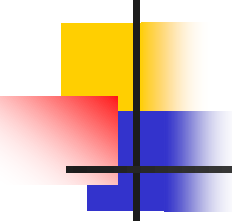
- Each process needs **minimum number of frames**
 - defined by the **instruction-set architecture**
 - Must have enough frames to hold all the different pages that any **single instruction can reference**.
- **Example: IBM 370 – 6 pages to handle SS MOVE instruction:**
 - instruction is 6 bytes, might span 2 pages.
 - 2 pages to handle **from**.
 - 2 pages to handle **to**.
- **Maximum number** is defined by the amount of available physical memory





10.5.2 Allocation Algorithms

- **Way to split frames among processes:**
 - **Equal allocation**
 - e.g., if 100 frames and 5 processes, give each 20 pages.
 - **Proportional allocation**
 - allocate available memory to each process according to its size
 - Use a **proportional allocation** scheme using **priorities** rather than size as the ratio of frames, or a combination of size and priority.



Proportional Allocation

s_i = size of process p_i

$$S = \sum s_i$$

m = total number of frames

$$a_i = \text{allocation for } p_i = \frac{s_i}{S} \times m$$

$$m = 64$$

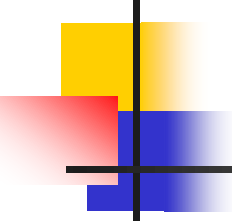
$$s_1 = 10$$

$$s_2 = 127$$

$$a_1 = \frac{10}{137} \times 64 \approx 5$$

$$a_2 = \frac{127}{137} \times 64 \approx 59$$





10.5.3 Global vs. Local Allocation

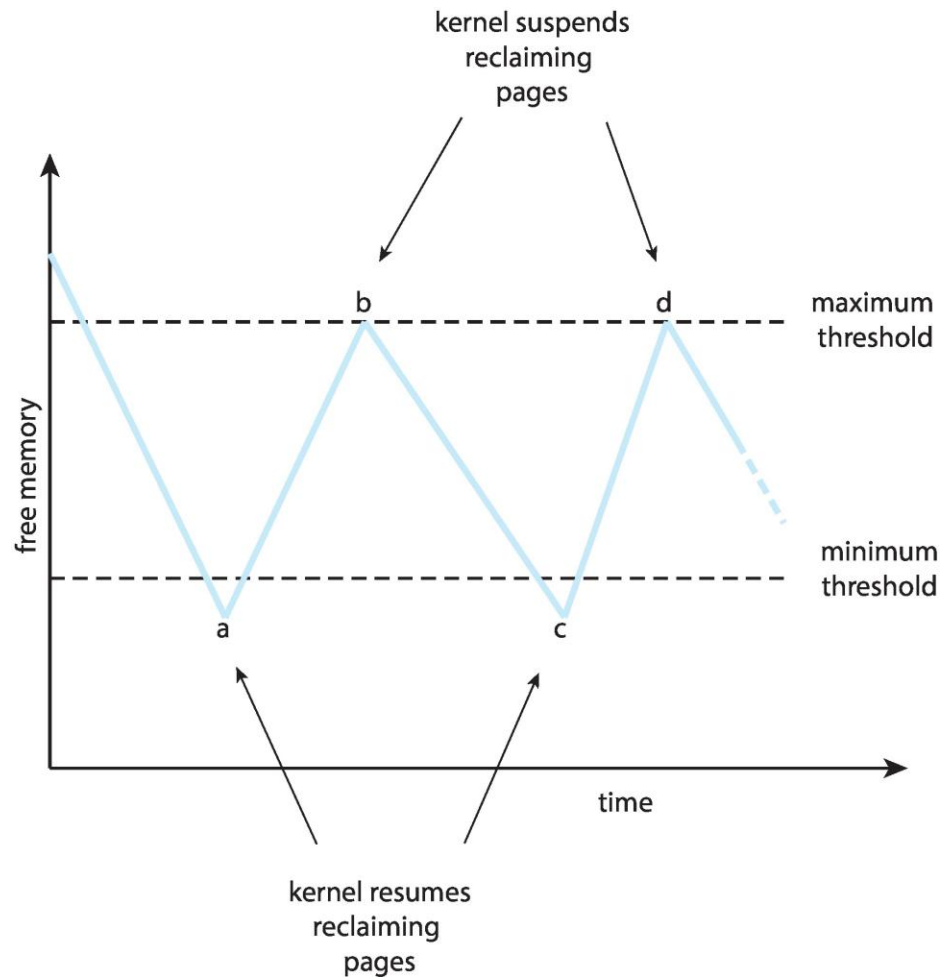
■ **Global replacement**

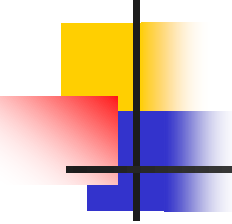
- **Process selects a replacement frame from the set of all frames; one process can take a frame from another.**
- **Allows high-priority processes to select frames from low-priority processes for replacement**
- **A process can not control its own page-fault rate!!**

■ **Local replacement**

- **Each process selects from only its own set of allocated frames**
- **Affected by the paging behavior of only that process.**

Reclaiming pages (Fig. 10.18)



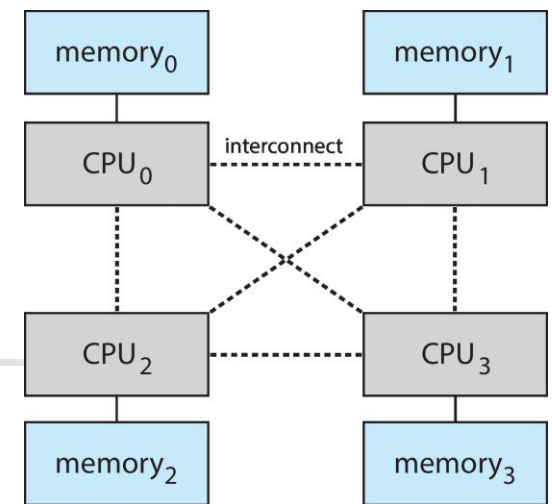


Global vs. Local Allocation (Cont.)

- Global replacement generally results in greater system throughput
 - therefore is the more common method

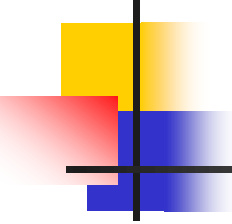


10.5.4 Non-Uniform Memory Access



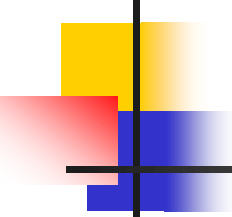
- Many systems are NUMA – speed of access to memory varies
 - Consider system boards containing CPUs and memory, interconnected over a system bus
- Optimal performance comes from allocating memory “close to” the CPU on which the thread is scheduled
 - modifying the scheduler to schedule the thread on the same system board when possible
 - Solved by Solaris by creating **lgroups**
 - Structure to track CPU / Memory low latency groups
 - Used by schedule and pager
 - When possible schedule all threads of a process and allocate all memory for that process within the lgroup





10.6 Thrashing

- If a process does not have “enough” frames, the page-fault rate is very high. This leads to:
 - low CPU utilization.
 - OS thinks that it needs to increase the degree of multiprogramming.
 - another process added to the system.
- **Thrashing** $\equiv$ a process is busy swapping pages in and out.



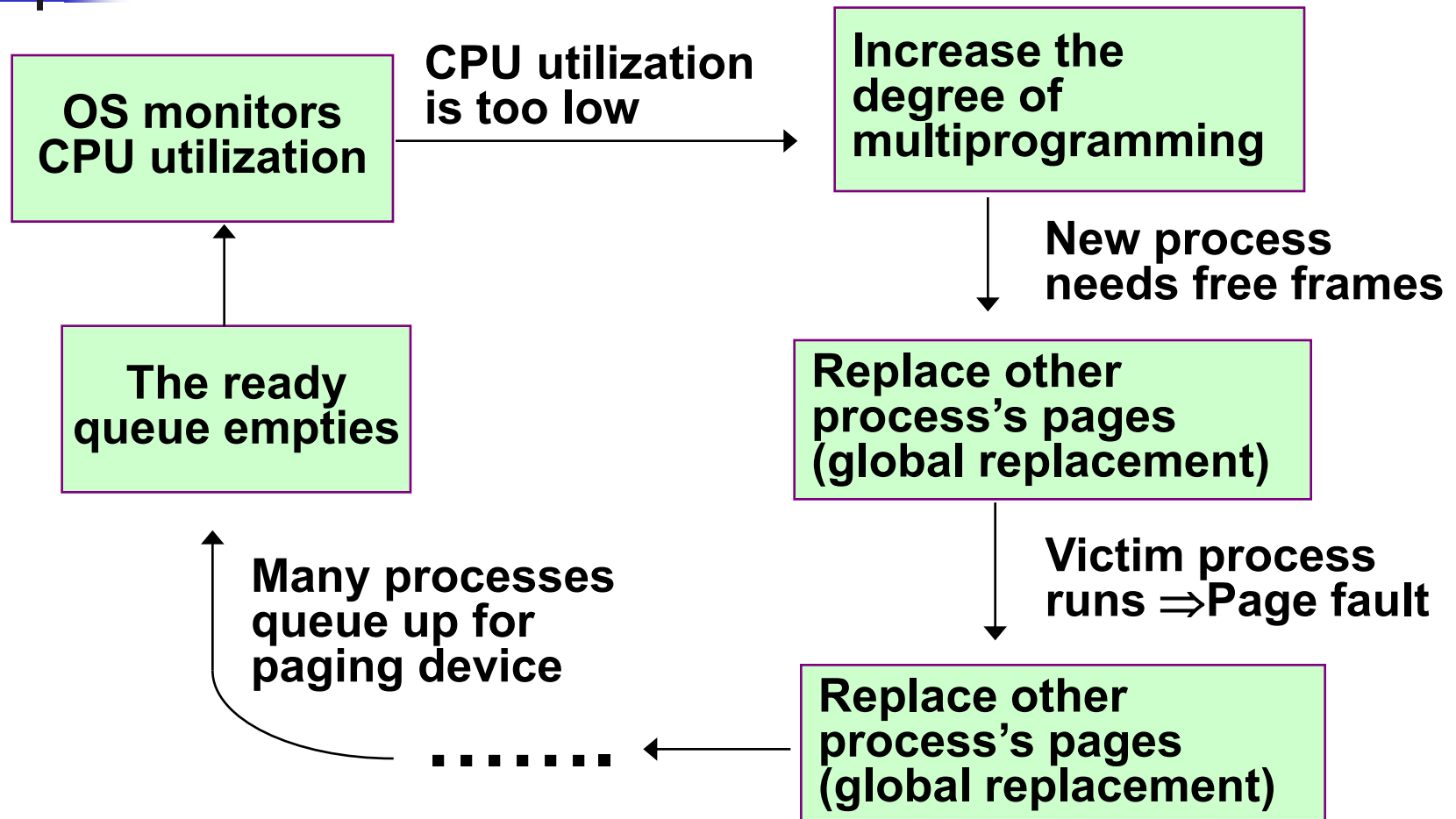
Thrashing (Cont.)

猛移現象; 輾轉現象

Any process does not have “enough” frames...

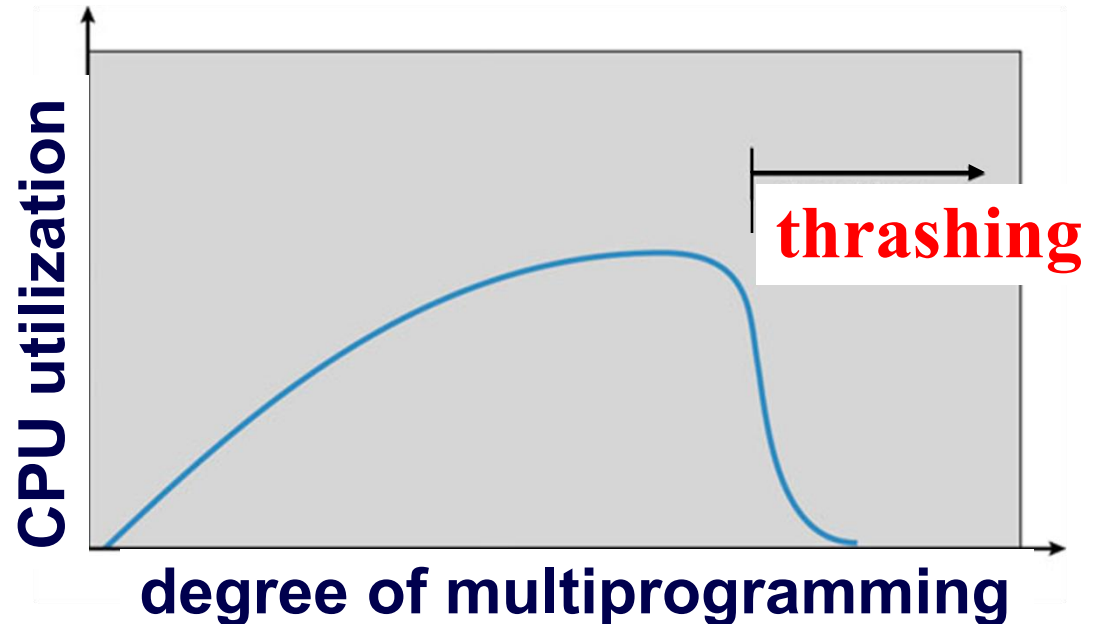
- If the process does not have the number of frames it needs to support pages in active use
 - it will quickly page-fault and replace some page
- Since all its pages are in active use, it must replace a page that will be needed again right away
- It quickly faults again, and again, and again, replacing pages that it must bring back in immediately.

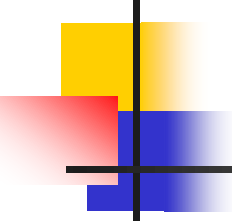
A Thrashing Scenario



Thrashing (Fig. 10.20)

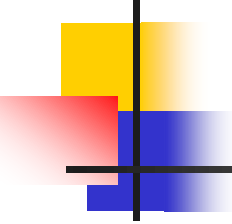
- Very high paging activity
- A process is *thrashing* if it is spending more time paging than executing.





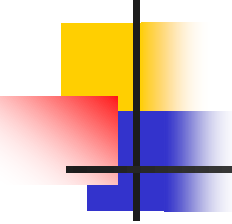
Overcoming Thrashing

- When the degree of multiprogramming is increased but CPU utilization drops
 - ☞ we should decrease the degree of multiprogramming
- Thrashing can be limited by using a **local (or priority) replacement algorithm**, however
 - ☞ Average page fault service time will increase, due to the longer average queue for the paging device
 - ☞ Effective access time will increase even for a process that is not thrashing



Overcoming Thrashing (Cont.)

- To prevent thrashing, we must provide a process as many frames as it needs
 - *working-set strategy* can be used to determine how many frames a process is actually using
 - Defines the *locality model* of process execution



Locality Model

- ***Locality model***

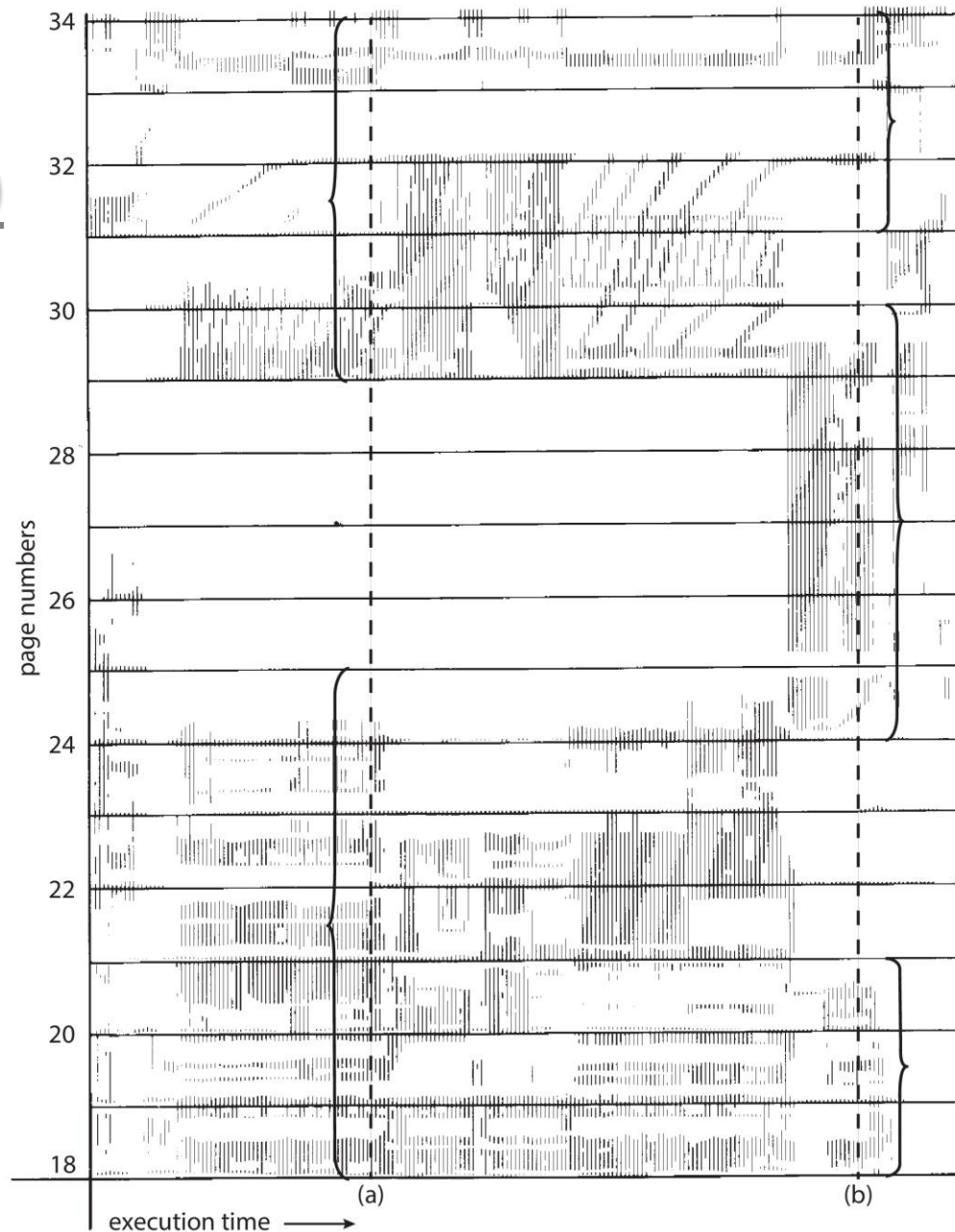
- Process migrates from one locality to another as a process executes.
- ***Locality*** - A set of pages that are actively used together
- Localities are defined by the program structure and its data structures. e.g., subroutine, array,...
- Localities may overlap.

- If we allocate fewer frames than the size of current locality, the process will thrash

- So, why does **thrashing** occur?
 Σ size of locality > total memory size

Locality In A Memory-Reference Pattern

(Fig. 10.21)

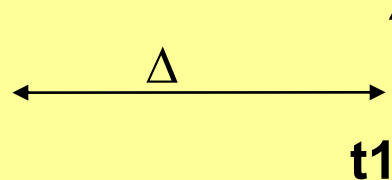


10.6.2 Working-Set Model

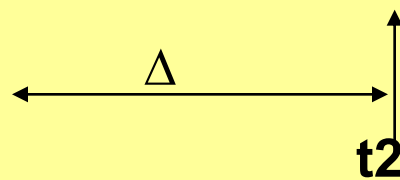
- Based on the assumption of locality
- Δ : **working-set window** $\equiv$ a fixed number of page references e.g., 10,000 instruction
- **working-set**: the set of pages in the most recent Δ page references (Fig. 9.20)

page reference table

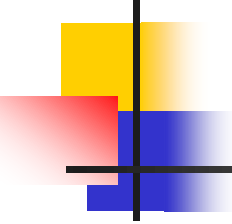
... 2 6 1 5 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 4 3 4 4 ...



$WS(t_1) = \{1, 2, 5, 6, 7\}$



$WS(t_2) = \{3, 4\}$



Working-Set Model (Cont.)

- If a page is in active use, it will be in the working set
- Working set is an approximation of the program's locality
- The selection of Δ is very important
 - Δ **too small**: not encompass the entire locality
 - Δ **too large**: may overlap several localities
 - $\Delta \rightarrow \infty$: working set = entire program

Use Working Set Model to Prevent Thrashing

- **Working-set size (WSS): number of pages in a WS**

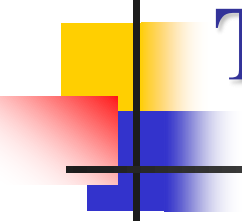
$D = \sum WSS_i$ (total demand frames)

If $D > m$ (total number of available frames)

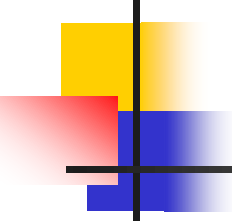
⇒ **Thrashing**

- **The use of the working-set model:**
 - OS monitors the WS of each process and allocates to that WS enough frames to provide it with its WS size.
 - When $D < m$, another process can be initiated
 - When $D > m$, OS selects a process to suspend. The process's pages are written out and its frames are reallocated to other processes. The suspended process can be restarted later.

Use Working Set Model to Prevent Thrashing (Cont.)

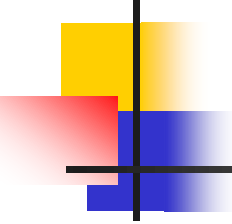


- **This working set strategy prevents thrashing while keeping the degree of multiprogramming as high as possible**
 - **Optimizes CPU utilization**



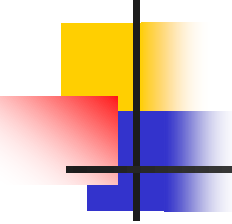
Working Set Implementation

- **Difficulty**: keep track of the working set (it is a moving window)
- **Approximate** with a fixed interval timer interrupt and a reference bit.
- **Example**: $\Delta = 10,000$ references
 - 1 timer interrupts every 5,000 references,
 - Keep in memory 2 bits for each page.
 - Whenever a timer interrupts, copy and clear the reference bits for all pages.
 - Those pages with at least 1 bit on will be considered to be in the working set



Working Set Implementation (Cont.)

- **Improvement:**
 - **Reduce the uncertainty:** more history bits and the frequency of interrupts.
 - e.g., 10 bits and interrupts every 1,000 references.



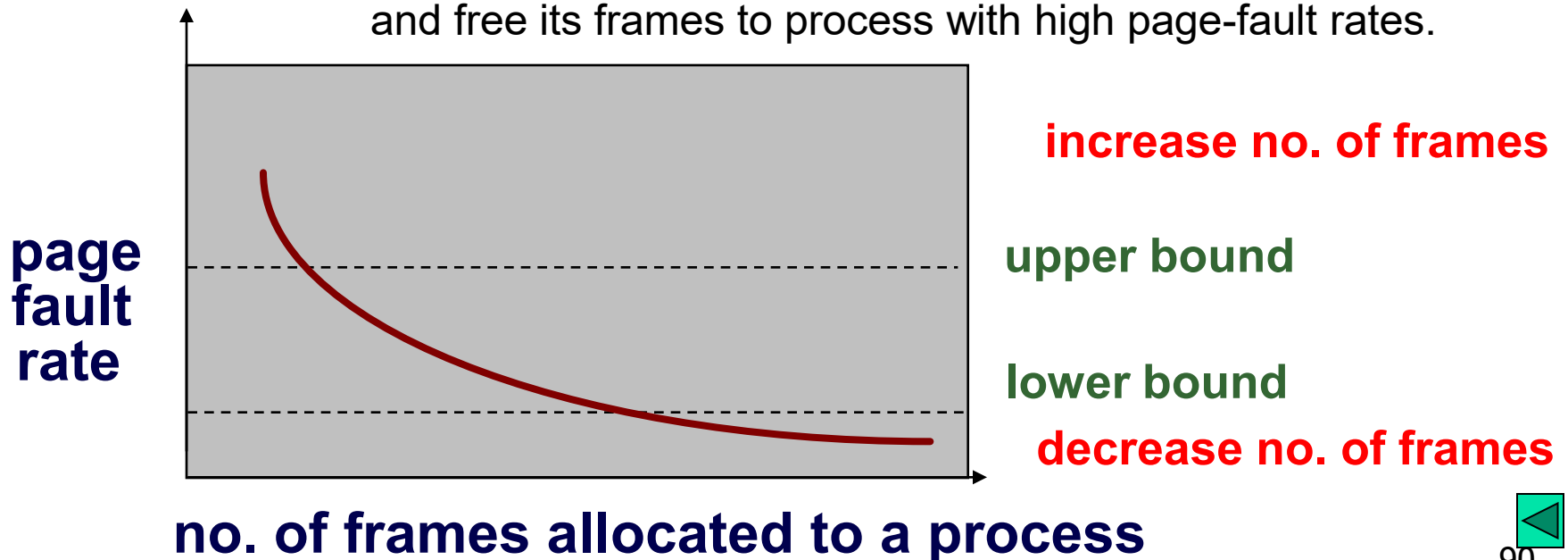
10.6.3 Page-Fault Frequency (PFF)

- The knowledge of the working set can be useful for **prepaging**, but it seems a rather clumsy way to control thrashing.
- **PFF takes a more direct approach.**
- Thrashing has a high page-fault rate. Thus, high page-fault rate needs more frames and low page-fault rate have too many frames.
- Establish **upper and lower bounds** on the desired page-fault rate.

Page-Fault Frequency (Cont.)

Directly measure and control page-fault rate to prevent thrashing.

- Establish “acceptable” page-fault rate.
 - If actual rate too low, process loses frame.
 - If actual rate too high, process gains frame.
 - If no free frame available, then select a process to suspend and free its frames to process with high page-fault rates.



10.7 Memory Compression

■ Fig. 24 Free-frame list before compression

free-frame list

head → 7 → 2 → 9 → 21 → 27 → 16

modified frame list

head → 15 → 3 → 35 → 26

■ Fig. 25 Free-frame list after compression

free-frame list

head → 2 → 9 → 21 → 27 → 16 → 15 → 3 → 35

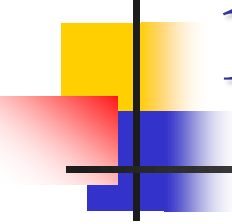
modified frame list

head → 26

compressed frame list

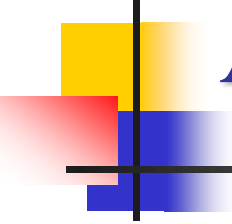
head → 7





10.8 Allocating Kernel Memory

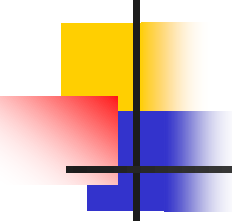
- **Kernel memory is often allocated from a free-memory pool different from the list used to satisfy ordinary user-mode processes:**
 - **Kernel may request memory for data structure of varying sizes**
 - should minimize waste due to fragmentation
 - **Certain hardware devices interact directly with physical memory**
 - Require allocated memory residing in physically contiguous pages.



Allocating Kernel Memory (Cont.)

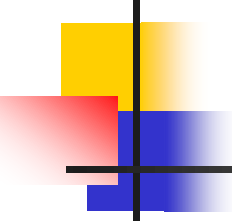
- **Two strategies for managing free memory assigned to kernel processes:**
 - **10.8.1 Buddy System** 
 - **10.8.2 Slab Allocation** 





10.8.1 Buddy System

- **Allocates memory from a fixed-size segment consisting of physically contiguous pages.**
- **Memory is allocated from this segment using a **power-of-2 allocator****
 - **Satisfies requests in units sized as a power of 2 (e.g. 4KB, 8KB, 16KB,...)**
 - **Request rounded up to next highest power of 2**
 - **When smaller allocation needed than is available, current chunk split into two buddies of next-lower power of 2**
 - **Continue until appropriate sized chunk available**



Buddy System (Cont.)

- **Advantage:**

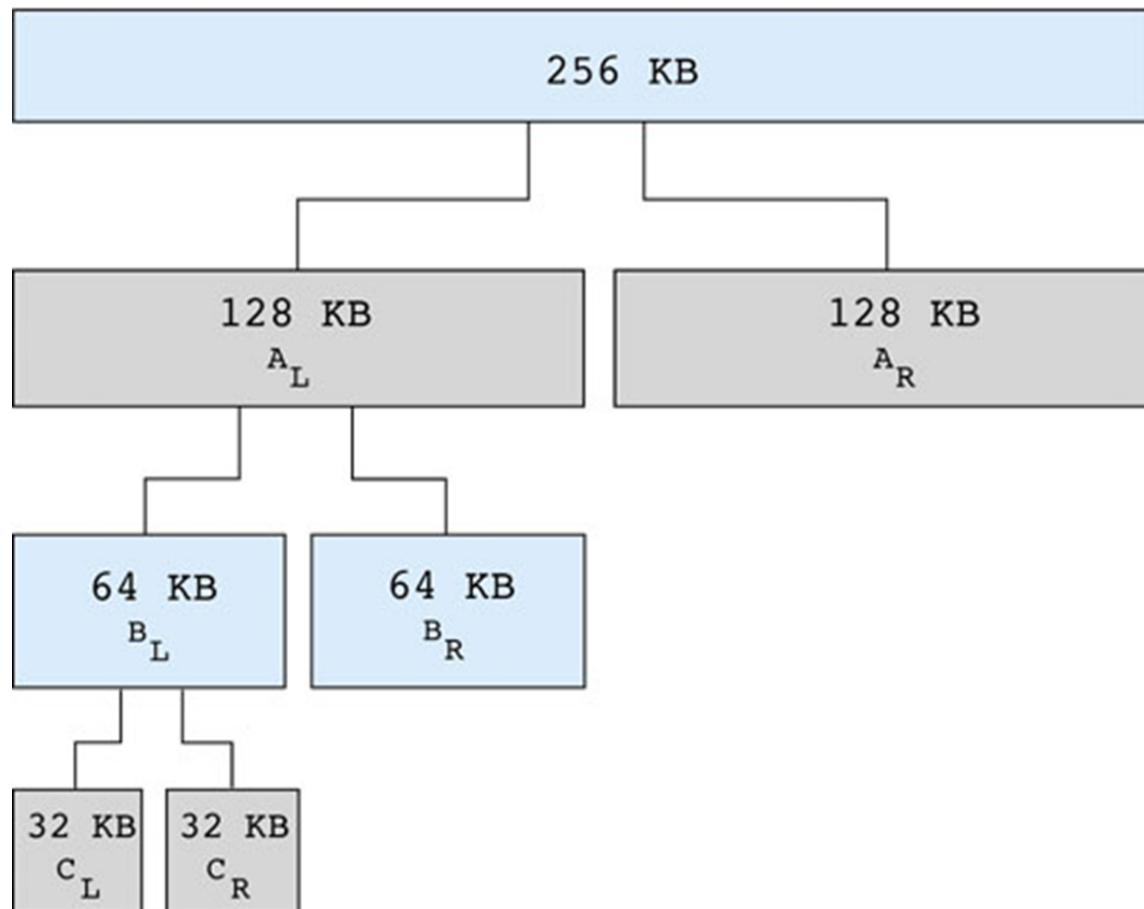
- adjacent buddies can be quickly combined to form larger segments using a technique known as coalescing 聯合,接合

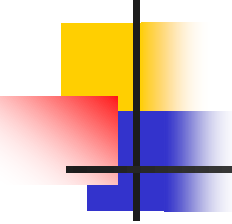
- **Drawback:**

- Internal fragmentation

Buddy System Allocation (Fig. 10.26)

physically contiguous pages

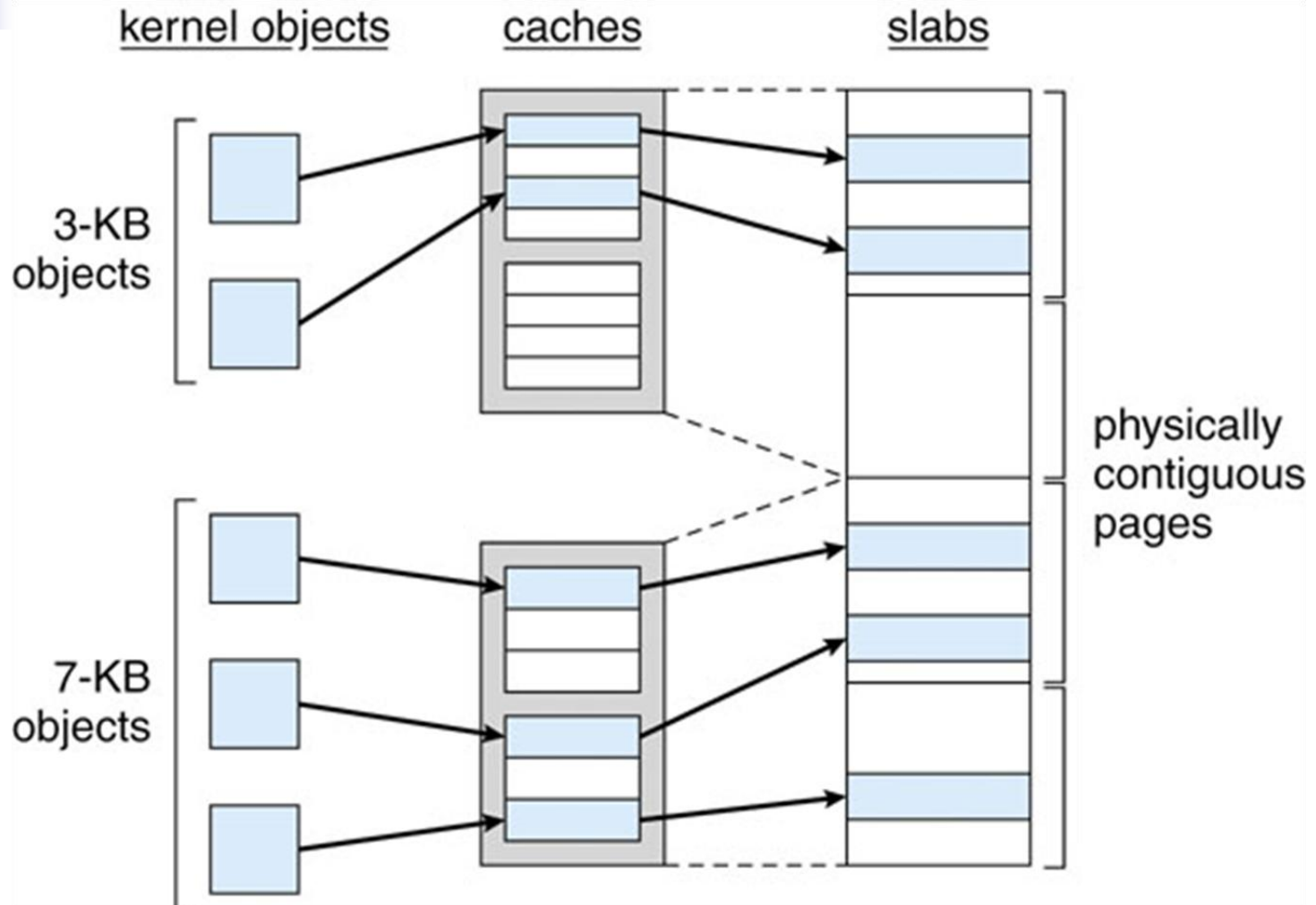


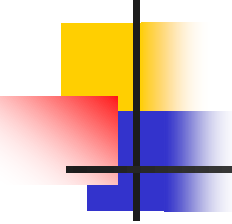


10.8.2 Slab Allocation

- A **slab** is made up of one or more physically contiguous pages.
- A **cache** consists of one or more slabs.
- A single cache for each unique kernel data structure
- Each cache is populated with **objects** that are instantiations of the kernel data structure the cache represents.
 - e.g. semaphore objects, process descriptor objects,...

Slab Allocation (Fig. 10.27)



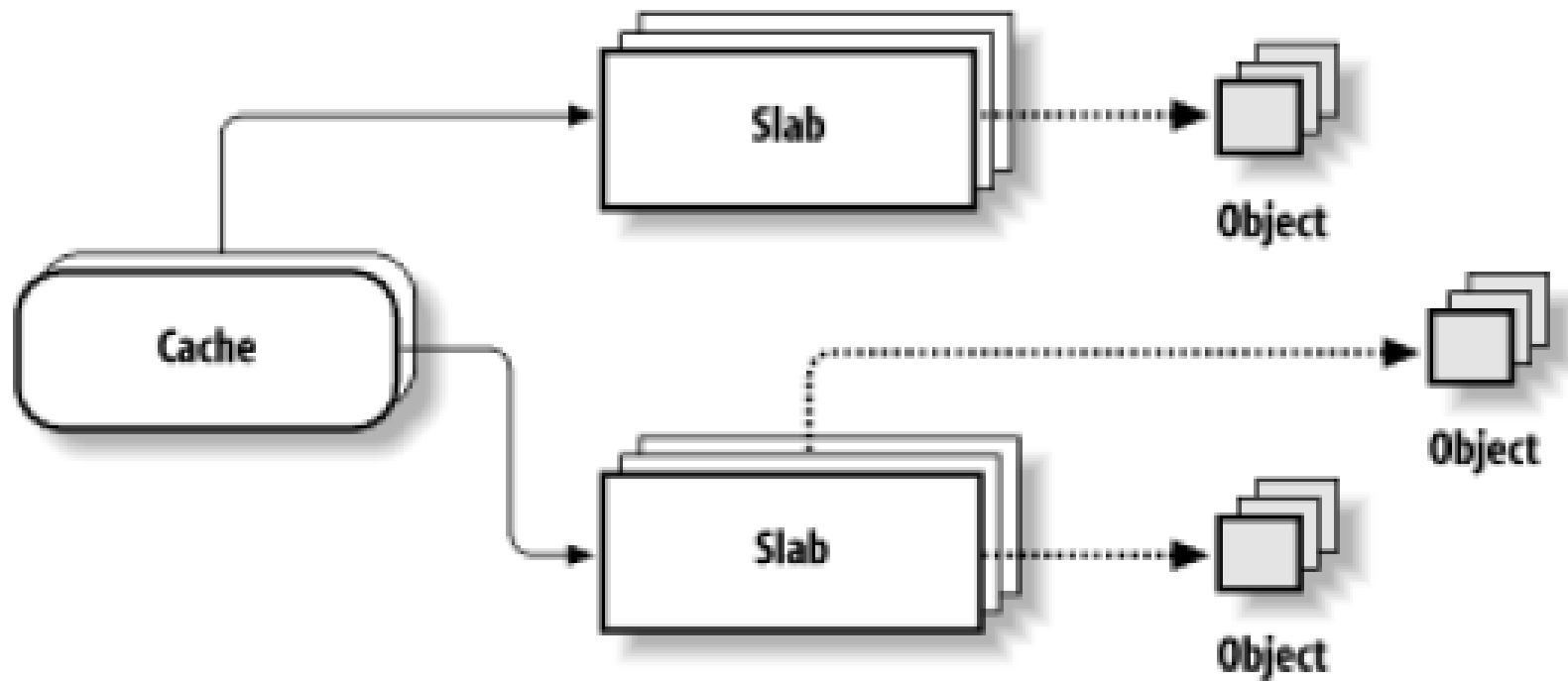


Slab Allocation (Cont.)

- **Two main benefits:**
 - **No memory is wasted due to fragmentation**
 - Each unique kernel data structure has an associated cache, and each cache is comprised of one or more slabs that are divided into chunks the size of the objects being represented.
 - **Memory requests can be satisfied quickly.**
 - Particularly effective for managing memory where objects are frequently allocated and deallocated
- **Solaris, Linux**

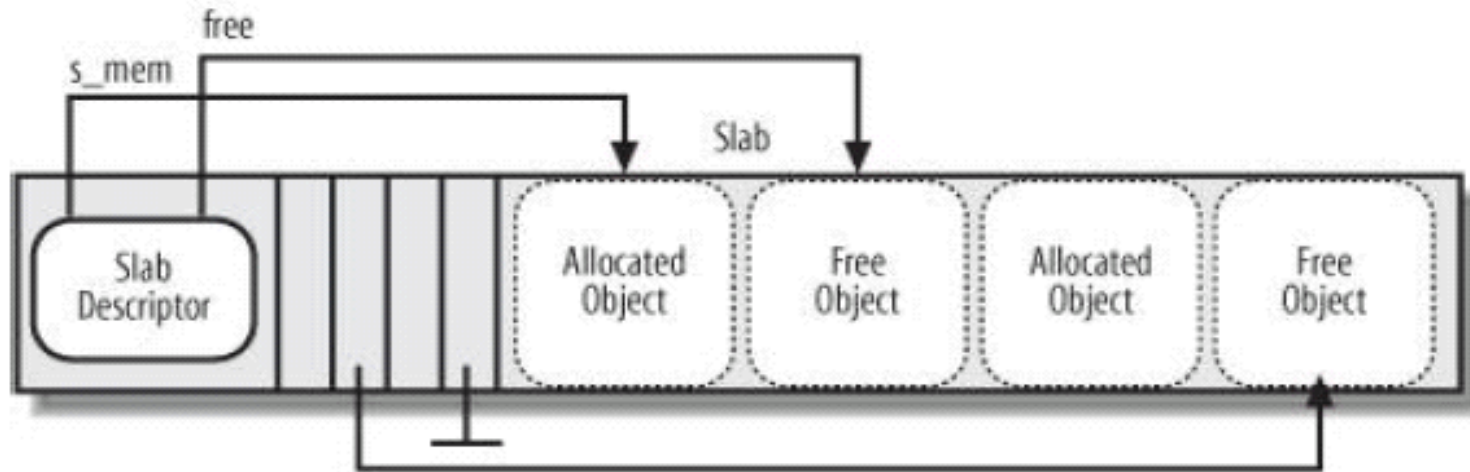


Slab Allocator Components

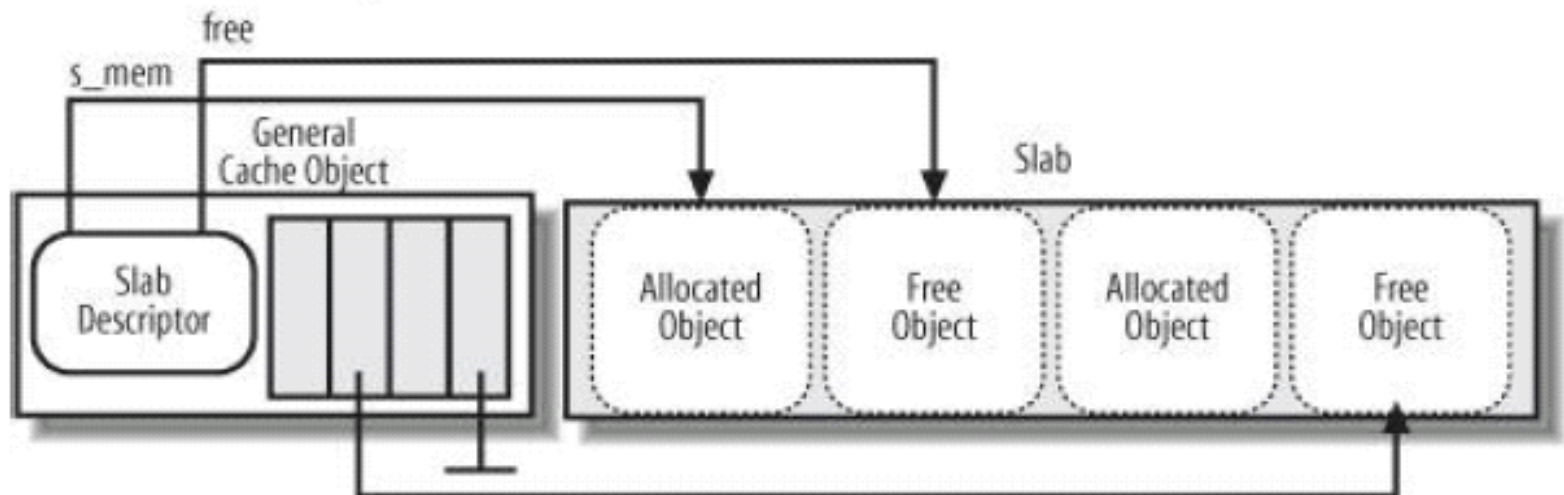


Slab and Object Descriptors

Slab with Internal Descriptors



Slab with External Descriptors



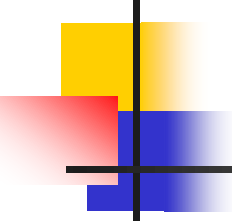
Data Structures for General Purpose Objects Allocation/Deallocation

kmalloc() & kfree() Linux 3.8.2

malloc_sizes[]

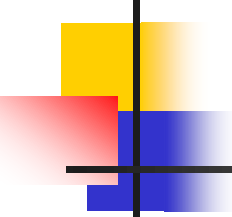
32
64
128
256
512
1024
2048
4096
8192
16384
32768
65536
131072
...
33554432

Cache descriptor of the
cache containing objects
of the right size.



10.9 Other Considerations

- 10.9.1 Prepaging 
- 10.9.2 Page Size 
- 10.9.3 TLB Reach 
- 10.9.4 Inverted Page Tables 
- 10.9.5 Program Structure 
- 10.9.6 I/O Interlock and Page Locking 

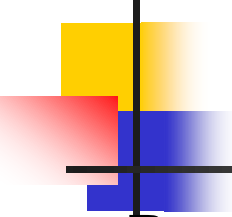


10.9.1 Prepaging

- With pure demand-paging, when a process starts or is paged in for restart, large number of page faults will occur
- *Prepaging*
 - Bring into memory at one time all the pages that will be needed (according to its working set)
 - Prevent the high level of initial page faults
 - Save the working set when a process is paged out
 - should consider

Cost of prepaging $\leq$ Cost of servicing page-faults?





10.9.2 Page Size Considerations

Page size selection must take into consideration:

- **Table size:**

- Decrease the page size increases the number of pages and hence the size of the page table

- **Fragmentation:**

- Memory is better utilized with smaller pages
- A small page size minimizes internal fragmentation

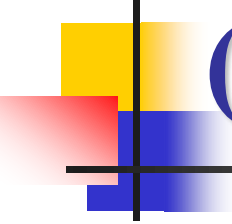
- **I/O overhead:**

- A desire to minimize I/O time argues for a larger page size

- **Locality:**

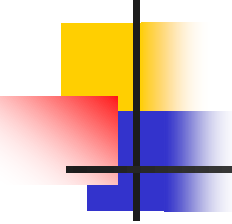
- A smaller page size allows each page to match program locality more accurately
- better resolution, allowing us to isolate only the memory that is actually needed
- less I/O, less total allocated memory

Page Size Considerations (Cont.)



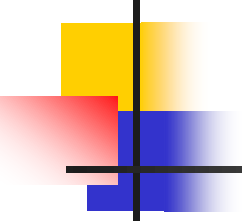
- A large page size minimizes the number of page faults
- Internal fragmentation and locality argue for a small page size
- Page table size and I/O time argue for a large page size
 - e.g., Intel 80386 has pages of 4K
- Systems allow different page sizes
 - e.g., IBM/370 allows pages of either 2K or 4K





10.9.3 TLB Reach

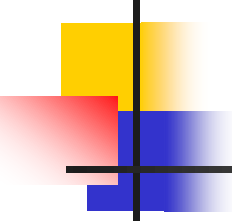
- A similar metric of the hit ratio of the TLB: **TLB reach**
- TLB Reach - The amount of memory accessible from the TLB.
 - $\text{TLB Reach} = (\text{number of entries in TLB}) \times (\text{Page Size})$
- Ideally, the working set for a process is stored in the TLB. If not, the process will spend a considerable amount of time resolving memory references in the page table rather than TLB.



Increasing the Size of the TLB

- **To increase the TLB reach:**
 - **Increase the number of entries in TLB**
 - **Increase the Page Size**
 - This may lead to an increase in fragmentation as not all applications require a large page size.
 - **Provide Multiple Page Sizes.**
 - This allows applications that require larger page sizes the opportunity to use them without an increase in fragmentation.





10.9.4 Inverted Page Tables

- Demand paging requires the location information to process page faults since a referenced page may not currently in memory.
- OS must additionally keep an external page table for each process to contain information on where each virtual page is located.



10.9.5 Program Structure

$A[128][128]$

$A[0][127]$

$A[127][0]$

assume page size
= 128 words

- `int A[ ][ ] = new int [128][128];`
- If OS allocates less than 128 frames to this program:

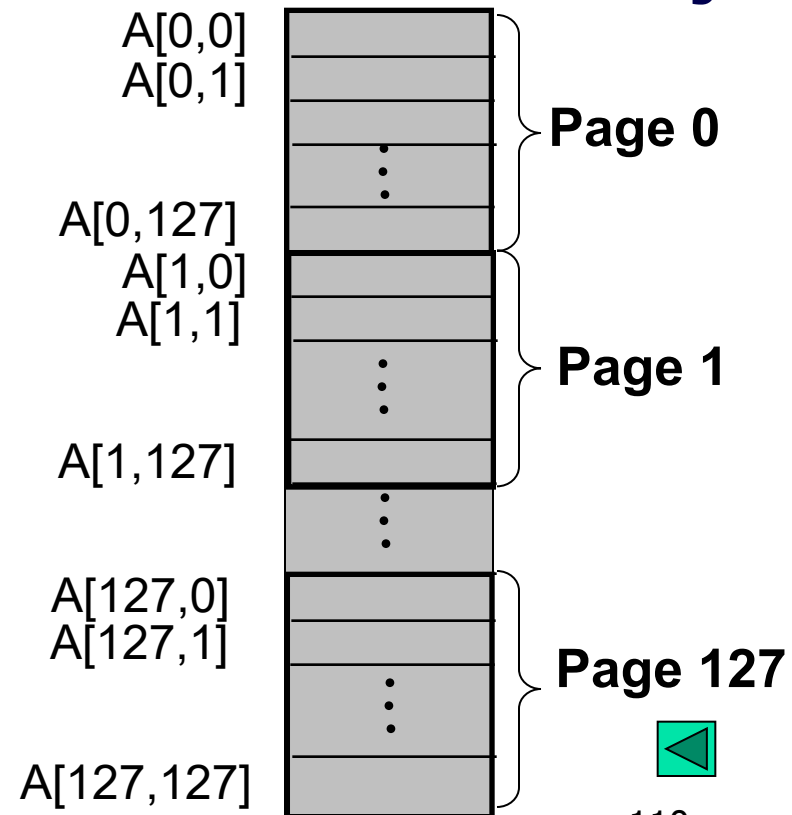
```
for (j = 0; j < A.length; j++)  
    for (i = 0; i < A.length; i++)  
        A[i,j] = 0;
```

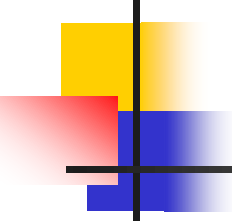
128 × 128 page faults

```
for (i = 0; i < A.length; i++)  
    for (j = 0; j < A.length; j++)  
        A[i,j] = 0;
```

128 page faults

Row major

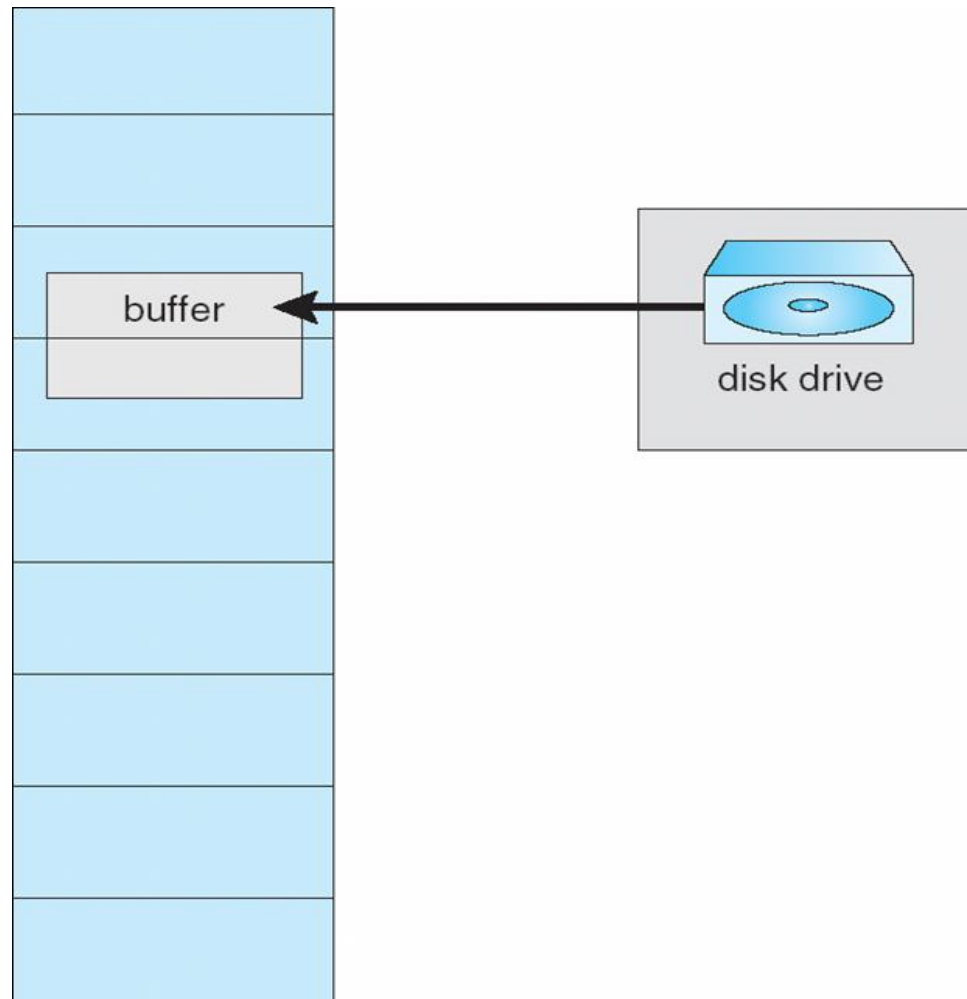


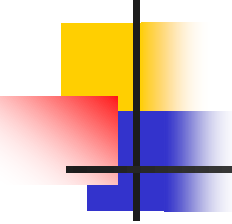


9.9.6 I/O Interlock

- Sometimes we want to allow some of the pages to be *locked* in memory (frames used for I/O).
- Methods:
 - ① Never to execute I/O to user memory
user memory $\Leftrightarrow$ system memory $\Leftrightarrow$ I/O device
 - ② Allow pages to be locked into memory
 - a **lock bit** is associated with every frame
 - a locked frame cannot be selected for replacement
- e.g. Macintosh, Solaris,...

Reason Why Frames Used For I/O Must Be In Memory (Fig. 10.28)

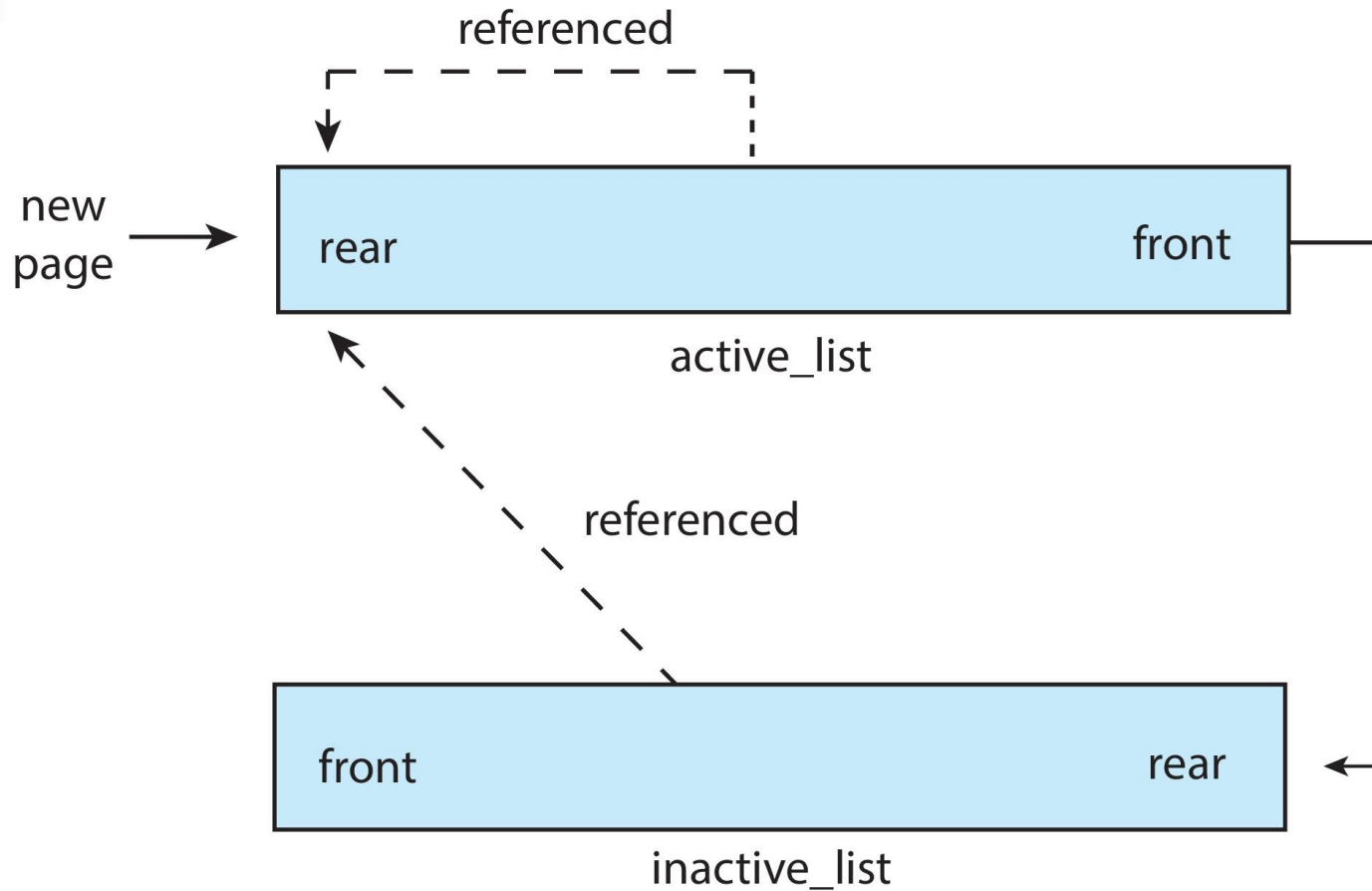


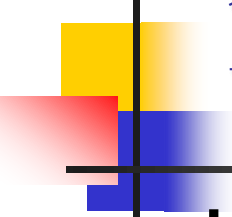


10.10 Operating-System Examples

- **10.10.1 Linux**
- **10.10.2 Windows**
- **10.10.2 Solaris**

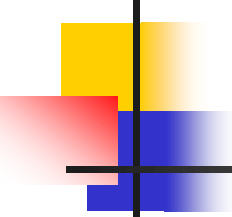
10.10.1 Linux





10.10.2 Windows

- Uses demand paging with **clustering**. Clustering brings in pages surrounding the faulting page
- Processes are assigned **working set minimum** and **working set maximum**
- Working set minimum is the minimum number of pages the process is guaranteed to have in memory
- A process may be assigned as many pages up to its working set maximum
- When the amount of free memory in the system falls below a threshold, **automatic working set trimming** is performed to restore the amount of free memory
- Working set trimming removes pages from processes that have pages in excess of their working set minimum



10.10.3 Solaris

- Maintains a list of free pages to assign faulting processes
- *Lotsfree* – threshold parameter (amount of free memory) to begin paging
- *Desfree* – threshold parameter to increasing paging
- *Minfree* – threshold parameter to being swapping
- Paging is performed by *pageout* process
- Pageout scans pages using modified clock algorithm
- *Scanrate* is the rate at which pages are scanned. This ranges from *slowscan* to *fastscan*
- Pageout is called more frequently depending upon the amount of free memory available
- Priority paging gives priority to process code pages

Solaris Page Scanner (Fig. 10.30)

